Supplementary Information: Comparative spatial analysis reveals important structural similarities between ancient Angkor and Late Anglo-Saxon Hampshire

2026-04-02

W. Christopher Carleton[1], Sarah Klassen[2], John T. Murphy[3], Patrick Roberts[1]

[1] *Department of Coevolution of Land Use and Urbanisation, Max Planck Institute of Geoanthropology, Jena, Germany* [2] *Center for Collaborative Synthesis in Archaeology, University of Colorado, Boulder, Colorado, USA*
[3] *Department of Anthropology, Northern Illinois University, DeKalb, Illinois, USA*

emails: carleton@gea.mpg.de, sarah.klassen@colorado.edu, jmurphy7@niu.edu, roberts@gea.mpg.de

Overview

This computational notebook contains the code necessary to rerun the analyses described in the associated paper. In order to gather and prepare the data used in this notebook for the Hampshire case, run the other computational notebook in the same folder ("doomsday.ipynb") first. Then, run all of the following code in order.

Set Up

Libraries

```python
# core
import numpy as np
import pandas as pd
from scipy.stats import norm
from pyproj import Transformer
import warnings
import os
from tqdm import tqdm
from scipy.signal import find_peaks
from scipy.spatial import ConvexHull

# Bayes
import pymc as pm
import pytensor.tensor as pt

# Plotting
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
import contextily as ctx
import arviz as az

# chronocluster
from chronocluster import clustering
from chronocluster.utils import (
    clustering_heatmap,
    pdiff_heatmap,
    get_box,
    chrono_plot,
    chrono_plot2d,
    inclusion_legend,
    plot_pdd
)
from chronocluster.distributions import ddelta
```

Plot Styling

```python
# basic styling
plt.style.use('ggplot')
sns.set_context("paper")
```

```
# matplotlib fonts
mpl.rcParams["font.size"] = 12
mpl.rcParams["legend.frameon"] = False
mpl.rcParams["legend.fontsize"] = 10
mpl.rcParams["axes.labelsize"] = 12
mpl.rcParams["axes.titlesize"] = 14
mpl.rcParams['figure.facecolor'] = 'white'
```

NOTE: Warnings

Some of the plotting workarounds have generated immaterial warnings below. In order to avoid these showing up in a PDF or HTML version of this notebook, like the one that will be submitted as SI alongside the associated paper, run the next cell. In order to see the warnings, remove the cell or change plot_warnings to True.

```
plot_warnings = False
if not plot_warnings:
    warnings.filterwarnings("ignore")
```

Angkor

Data Wrangling

```
# data wrangling
df = pd.read_csv('../Data/temples_with_predicted_ages.csv')
df = df.dropna(subset=['xeast', 'ynorth', 'model_age_mean'])
```

```
# data wrangling
df = pd.read_csv('../Data/temples_with_predicted_ages.csv')

# --- Step 1: inspect missingness ---
print("Initial count:", len(df))
print("Missing xeast:", df['xeast'].isna().sum())
print("Missing ynorth:", df['ynorth'].isna().sum())

# --- Step 2: build transformer (WGS84 → UTM zone for Angkor, likely 48N) ---
transformer = Transformer.from_crs("EPSG:4326", "EPSG:32648", always_xy=True)
```

```python
# --- Step 3: identify rows where xeast/ynorth missing but lon/lat present
 ↪  ---
mask_missing_xy = df['xeast'].isna() | df['ynorth'].isna()
mask_has_lonlat = df['xlong'].notna() & df['ylat'].notna()

recover_mask = mask_missing_xy & mask_has_lonlat

print("Recoverable from lon/lat:", recover_mask.sum())

# --- Step 4: convert lon/lat → UTM and fill ---
lon = df.loc[recover_mask, 'xlong'].values
lat = df.loc[recover_mask, 'ylat'].values

xeast, ynorth = transformer.transform(lon, lat)

df.loc[recover_mask, 'xeast'] = xeast
df.loc[recover_mask, 'ynorth'] = ynorth

# --- Step 5: final filtering ---
df = df.dropna(subset=['xeast', 'ynorth', 'model_age_mean'])

print("Final count after recovery + filtering:", len(df))
```

```
Initial count: 1431
Missing xeast: 542
Missing ynorth: 542
Recoverable from lon/lat: 397
Final count after recovery + filtering: 1286
```

Point Density

We derive a simple density estimate for temples at Angkor by assuming a minimum bounding hull for the area and then converting the temple point density to km^2.

```python
# Build coordinate array
coords = df[['xeast', 'ynorth']].to_numpy()

# Compute convex hull
hull = ConvexHull(coords)
```

```python
# Area in 2D: for scipy ConvexHull, 'volume' is the polygon area in 2D
hull_area_m2 = hull.volume
hull_area_km2 = hull_area_m2 / 1_000_000

# Point count and density
n_points = len(coords)
density_per_km2 = n_points / hull_area_km2

print(f"Angkor point count: {n_points}")
print(f"Convex hull area: {hull_area_m2:,.2f} m²")
print(f"Convex hull area: {hull_area_km2:,.2f} km²")
print(f"Point density: {density_per_km2:,.3f} points/km²")
```

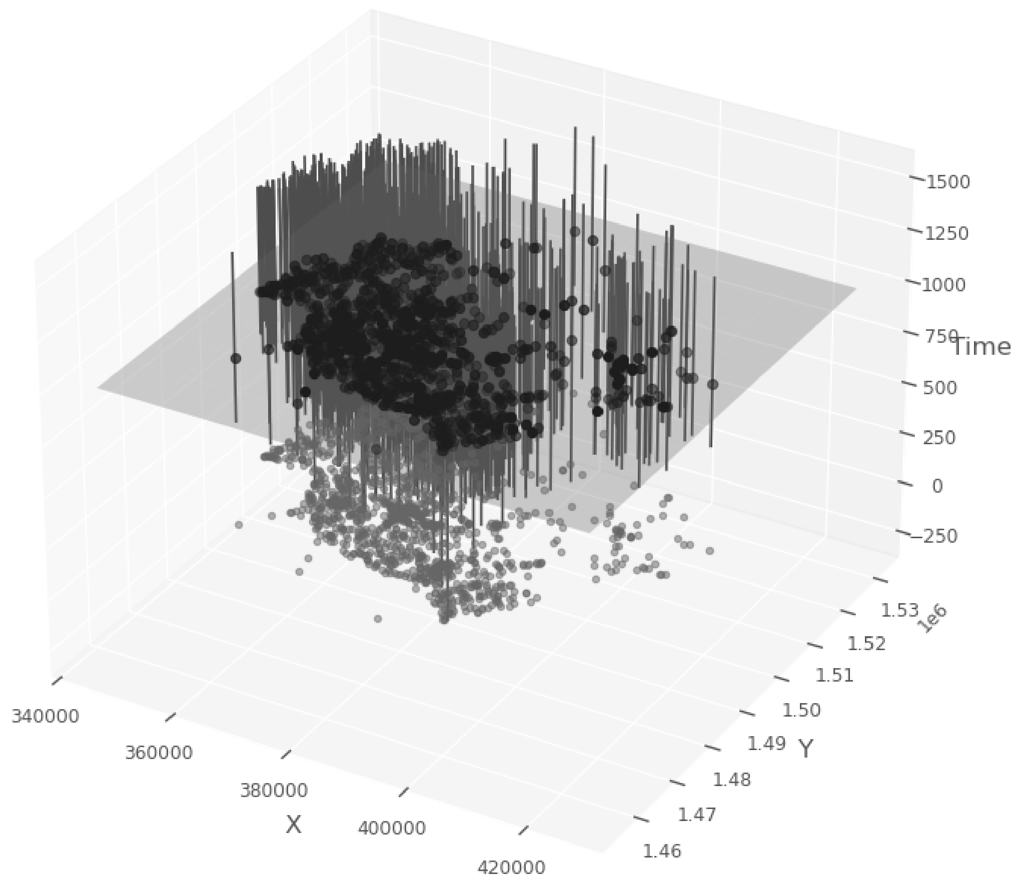
```
Angkor point count: 1286
Convex hull area: 3,089,638,360.59 m²
Convex hull area: 3,089.64 km²
Point density: 0.416 points/km²
```

Create Points List

```python
points = [
    clustering.Point(
        x=row['xeast'],
        y=row['ynorth'],
        start_distribution = (
            ddelta(d=row['model_age_mean'])
            if row['model_age_sd'] == 0
            else norm(loc=row['model_age_mean'], scale=row['model_age_sd'])
            ),
        end_distribution = ddelta(1500)
    )
    for _, row in df.iterrows()
]

# just double check the first ten look right
points[:10]
angkor_points = points
```

Figure S1: Spacetime Volume of Angkor's Temples

```
# Custom styling parameters
style_params = {
    'start_mean_color': None,  # Do not plot start mean points
    'end_mean_color': None, # Do not plot end mean points
    'mean_point_size': 10,
    'cylinder_color': (0.3, 0.3, 0.3),  # Dark grey
    'ppf_limits': (0.05, 0.95),  # Use different ppf limits
    'shadow_color': (0.4, 0.4, 0.4),  # grey
    'shadow_size': 10,
    'time_slice_color': (0.5, 0.5, 0.5),  # Grey
    'time_slice_alpha': 0.3,
    'time_slice_point_color': (0, 0, 0),  # Black
}

# Plot the points using the chrono_plot function with custom styling and a
# time slice plane
ax_stv_angkor, fig_stv_angkor = chrono_plot(points,
                                            style_params=style_params,
                                            time_slice=1000,
                                            title='Angkor')
ax_stv_angkor.set_box_aspect(None, zoom=0.85)
plt.savefig("../Output/spacetime_volume_Angkor.svg", bbox_inches='tight')
```



Define Time Slices

```
# Define the time slices
start_time = 800
end_time = 1200
time_interval = 50
time_slices = np.arange(start_time, end_time, time_interval)
time_slices
```

```
array([ 800,  850,  900,  950, 1000, 1050, 1100, 1150])
```

GPU Boosted Pairwise Distance Density KDEs

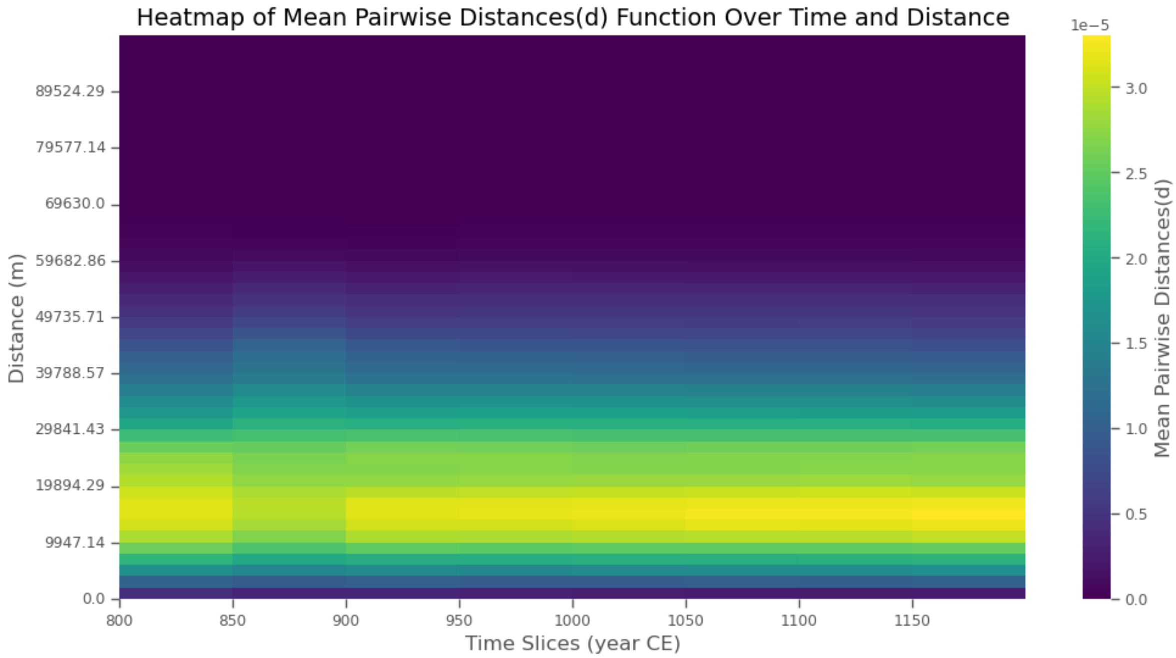
To speed up the production of the heatmap surfaces considerably, you can use GPU processing via the CUML library. If, however, you do not have access to a dedicated, compatiible GPU, set 'kde_custom=cuml_kde' to 'kde_custom=None' in all the subsequent cells where appropriate. Without the GPU acceleration, the following cells could take 15-20 minutes each to run, or longer, depending on the host system.

```
from cuml.neighbors import KernelDensity

def cuml_kde(distances, bandwidth, **kwargs):
    distances = np.array(distances).reshape(-1, 1)

    if bandwidth is None:
        n = len(distances)
        if n < 2:
            raise ValueError(
                "Data must contain at least 2 points ",
                "for bandwidth calculation."
            )
        std_dev = np.std(distances, ddof=1)
        bandwidth = std_dev * n ** (-1 / 5)

    kde = KernelDensity(kernel="gaussian", bandwidth=bandwidth, **kwargs)
    kde.fit(distances)

    def kde_function(points):
        points = np.array(points).reshape(-1, 1)
        # score_samples returns a cupy array; use .get() to convert to NumPy
        return np.exp(kde.score_samples(points).get())

    return kde_function
```

Figure S2: Heatmap of Pairwise Distance Density versus Time at Angkor

```
# Run the Monte Carlo simulation to get an ensemble of probable
# lists of points included in each time slice.
num_iterations = 500
simulations_angkor = clustering.mc_samples(
```

```
    points,
    time_slices=time_slices,
    num_iterations=num_iterations
)

# Get a bounding box for use later and to extract sensible distance limits
x_min, y_min, x_max, y_max = get_box(points)
max_distance = np.ceil(np.sqrt((x_max - x_min)**2 + (y_max - y_min)**2))

# set consistent pairwise bandwidth (binning of distances)
use_kde = True
pair_bw = None
kde_sample_n = 50
# here set kde_custom=None to use a CPU instead of GPU for processing.
kde_custom=cuml_kde

# Produce pairwise distances to explore clustering structure
pairwise_density_angkor, support_angkor = clustering.temporal_pairwise(
    simulations_angkor,
    time_slices,
    bw=pair_bw,
    use_kde=use_kde,
    kde_sample_n=kde_sample_n,
    max_distance=max_distance,
    kde_custom=kde_custom
)

# Visualize clustering with heatmap
fig, ax = clustering_heatmap(
    pairwise_density_angkor,
    support_angkor,
    time_slices,
    result_type='Pairwise Distances'
    #save = "../Output/pdd_hm_angkor.png"
)

ax.set_ylabel("Distance (m)")
ax.set_xlabel("Time Slices (year CE)")

plt.savefig("../Output/pdd_hm_angkor.svg", bbox_inches='tight')
plt.savefig("../Output/pdd_hm_angkor.png", bbox_inches='tight')
```



Complete Spatial Randomness

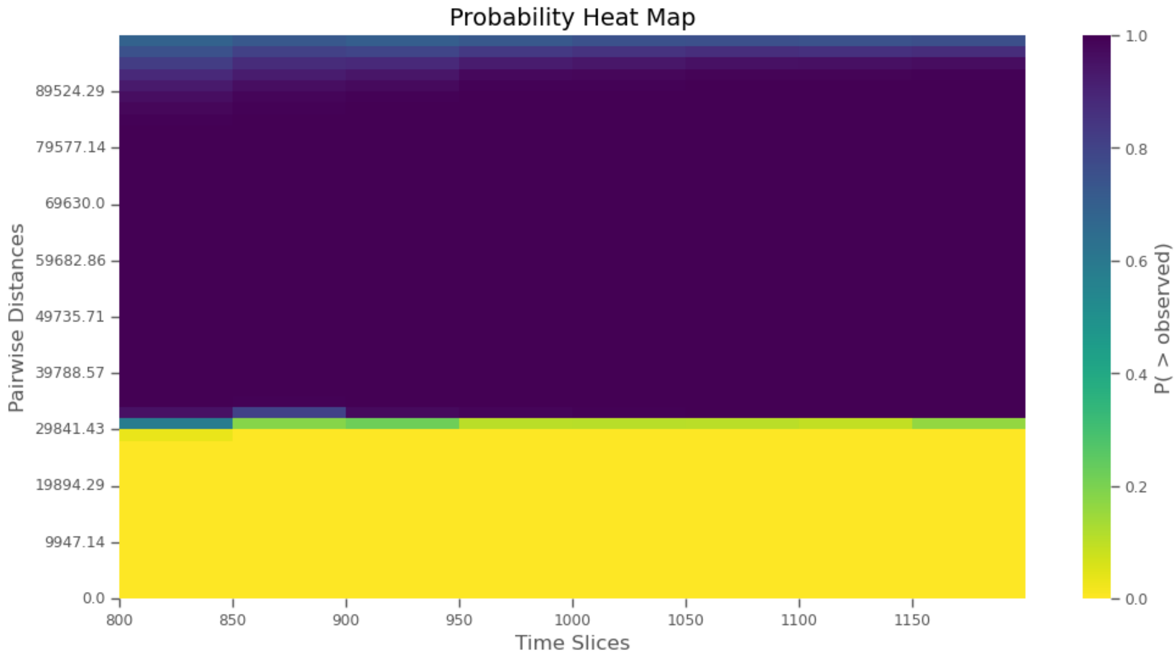
Figure S3: Heatmap of Pairwise Distance Density versus Time for CSR based on Angkor's Temples

```
# Get MC iterations for incorporating chronological uncertainty and CSR
csr_simulations_angkor = clustering.mc_samples(
    points,
    time_slices = time_slices,
    num_iterations = num_iterations,
    null_model=clustering.csr_sample,
    x_min=x_min,
    x_max=x_max,
    y_min=y_min,
    y_max=y_max
)

# Calulate the pairwise distances for the CSR sample
csr_pairwise_density_angkor, csr_support_angkor =
↪   clustering.temporal_pairwise(
    csr_simulations_angkor,
```

```
    time_slices,
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n=kde_sample_n,
    max_distance = max_distance,
    kde_custom=kde_custom
)

# Visualize clustering with heatmap
fig, ax = clustering_heatmap(
    csr_pairwise_density_angkor,
    csr_support_angkor,
    time_slices,
    result_type='Pairwise Distances'
)

ax.set_ylabel("Distance (m)")
ax.set_xlabel("Time Slices (year CE)")
```

Figure S4: Heatmap of PDD Statistical Significance compared to CSR for Angkor's Temples

```
# Calculate the p-values for density differences between the observed points
 ↪  and
# the simulated CSR baseline per distance and temporal slice
p_diff_array_csr_angkor, diff_array_csr_angkor = clustering.p_diff(
    pairwise_density_angkor,
    csr_pairwise_density_angkor
)

# Plot the heatmap of probabilities
fig, ax = pdiff_heatmap(
    p_diff_array_csr_angkor,
    time_slices,
    csr_support_angkor
)

ax.set_ylabel("Distance (m)")
ax.set_xlabel("Time Slices (year CE)")
```



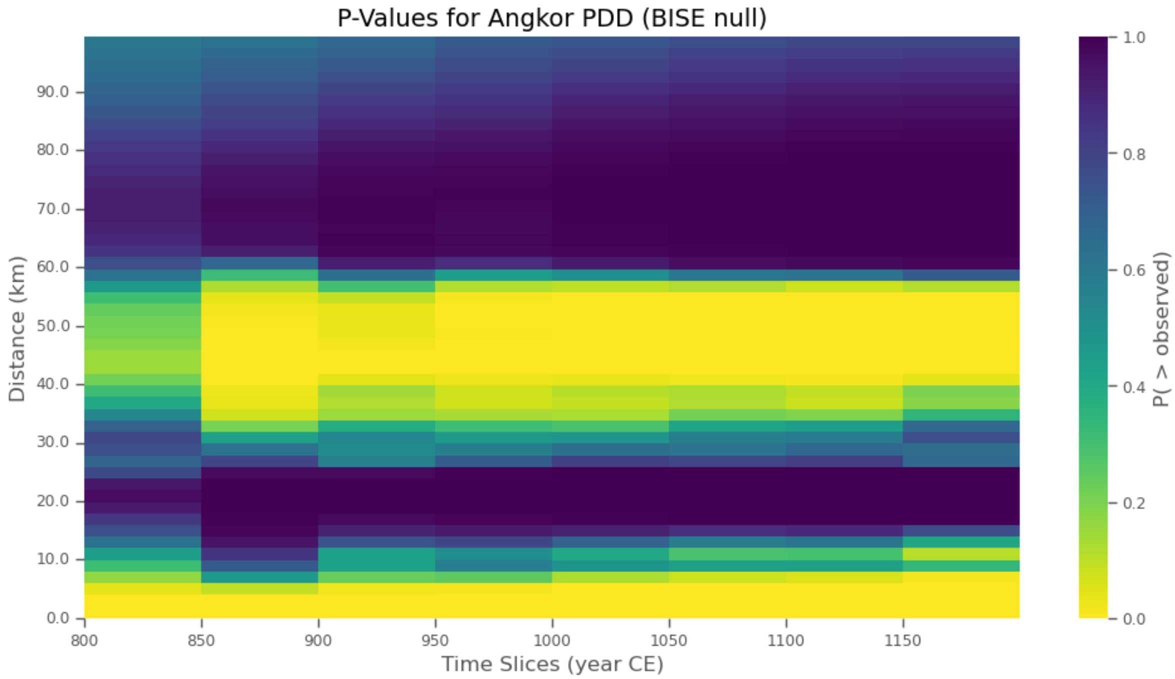
Baseline-Informed Spatial Expectation

Figure S5: Heatmap of PDD statistical significance compared to BISE based on Angkor's Temples

```
# Get MC iterations for incorporating chronological uncertainty with BISE
bise_simulations_angkor = clustering.mc_samples(points,
                                                 time_slices,
                                                 num_iterations = num_iterations,
                                                 null_model = clustering.bise)

# Calulate the pairwise distances for the LISE sample
bise_pairwise_density_angkor, bise_support_angkor =
↪   clustering.temporal_pairwise(
    bise_simulations_angkor,
    time_slices,
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n = kde_sample_n,
    max_distance = max_distance,
    kde_custom = kde_custom
```

```
)

# Calculate the p-values for density differences between the observed points
 ↪  and
# the simulated BISE baseline per distance and temporal slice
p_diff_array_bise_angkor, diff_array_bise_angkor = clustering.p_diff(
    pairwise_density_angkor,
    bise_pairwise_density_angkor
)
```

```
# Plot the heatmap of probabilities
fig, ax = pdiff_heatmap(
    p_diff_array_bise_angkor,
    time_slices,
    bise_support_angkor
)

# Custom ticks and labels here
tick_labels_km = np.arange(0, bise_support_angkor.max() / 1000, 10)
tick_labels_m = tick_labels_km * 1000
tick_positions = np.interp(
    tick_labels_m,
    bise_support_angkor,
    np.arange(len(bise_support_angkor))
)

ax.set_yticks(tick_positions)
ax.set_yticklabels(np.round(tick_labels_km, 1))
ax.set_ylabel("Distance (km)")

ax.set_xlabel("Time Slices (year CE)")
ax.set_title("P-Values for Angkor PDD (BISE null)")

plt.savefig("../Output/dpdd_hm_angkor.svg", bbox_inches='tight')
plt.savefig("../Output/dpdd_hm_angkor.png", bbox_inches='tight')
```



One Time Slice

Figure S6: Time Slice of PDD for Angkor compared to Null Models

```
time_slice_idx = np.where(time_slices == 1000)[0][0]

# List of density arrays
density_arrays = [
    pairwise_density_angkor,
    csr_pairwise_density_angkor,
    bise_pairwise_density_angkor
]

# Generate the plot and get the figure and axis objects
fig, ax = plot_pdd(
    time_slices=time_slices,
    time_slice_idx=time_slice_idx,
    support=support_angkor,
    density_arrays=density_arrays,
    quantiles=[0.025, 0.975],
    density_names=["Empirical", "CSR", "BISE"],
```

```
    colors=["blue", "orange", "green"]
)

ax.set_title("PDD Angkor 1000 CE")

# Get current tick positions and convert labels to km
x_ticks = ax.get_xticks()
ax.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax.set_xlabel("Distance (km)")

# Show the plot
plt.show()

fig.savefig("../Output/pdd_null_angkor.png", dpi=300, bbox_inches="tight")
fig.savefig("../Output/pdd_null_angkor.svg", bbox_inches="tight")
```

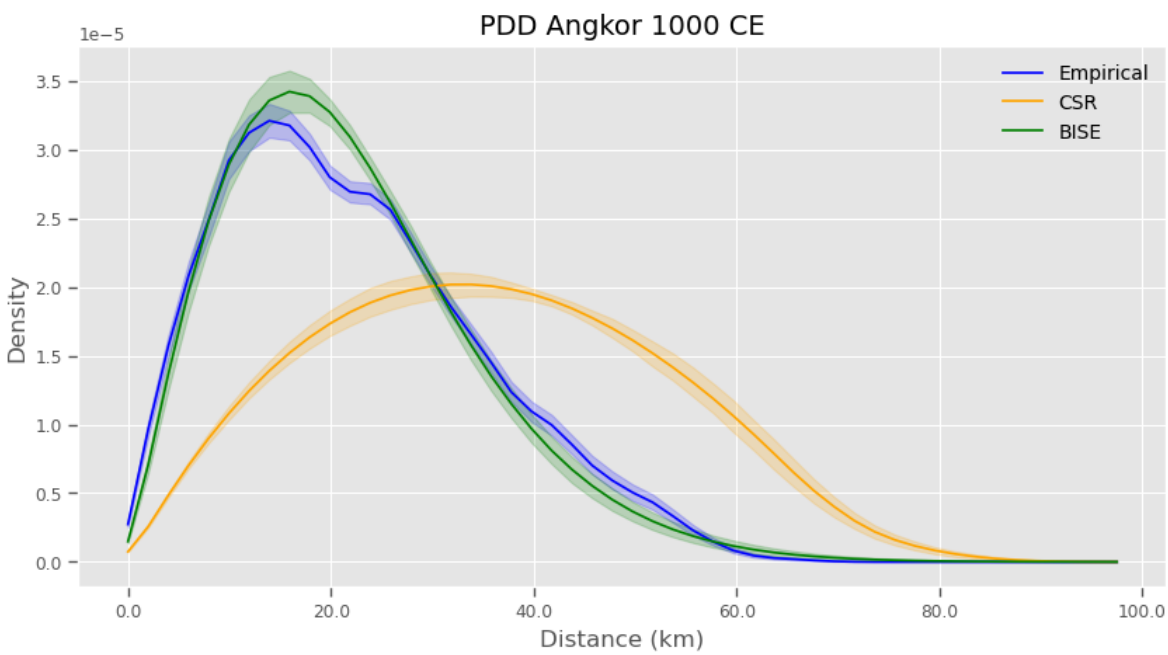
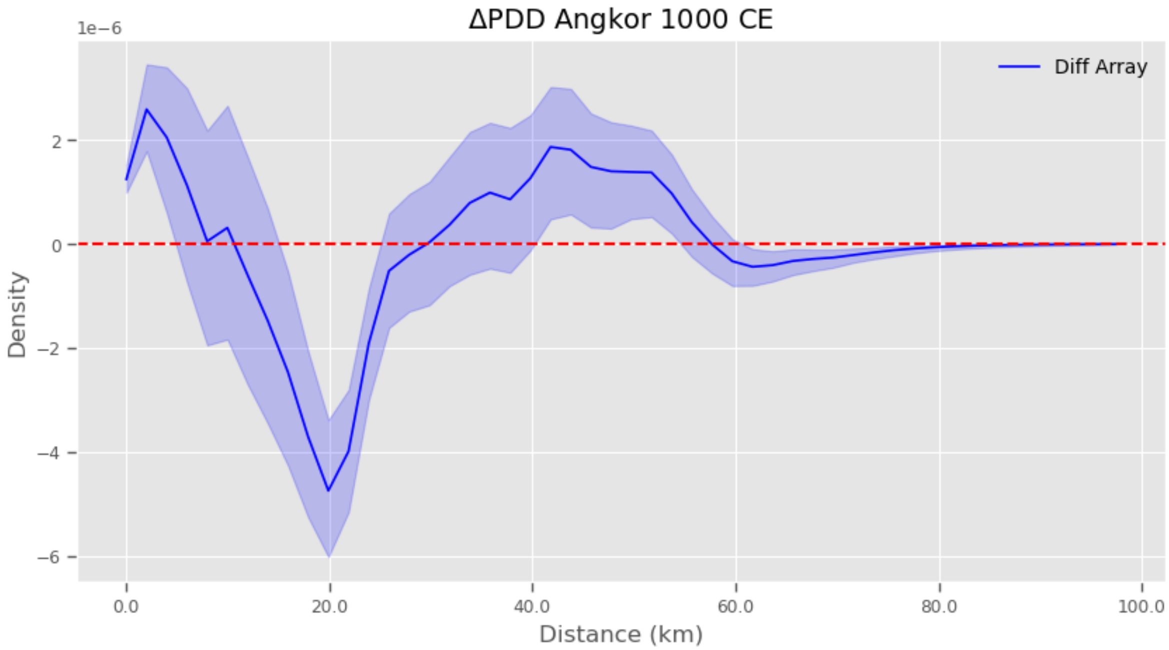


Figure S7: Difference between Angkor PDD and BISE Null at 1000 CE

```python
# List of density arrays
density_arrays = [diff_array_bise_angkor]

time_slice_idx = np.where(time_slices == 1000)[0][0]

# Generate the plot and get the figure and axis objects
fig1, ax1 = plot_pdd(
    time_slices=time_slices,
    time_slice_idx=time_slice_idx,
    support=support_angkor,
    density_arrays=density_arrays,
    quantiles=[0.025, 0.975],
    density_names=["Diff Array"],
    colors=["blue"]
)

# Add a horizontal line at y=0
ax1.axhline(y=0, color='red', linestyle='--', linewidth=1.5)

ax1.set_title("$\Delta$PDD Angkor 1000 CE")
#ax1.set_xlabel("Distance (m)")

# Get current tick positions and convert labels to km
x_ticks = ax1.get_xticks()
ax1.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax1.set_xlabel("Distance (km)")

# Show the plot
plt.show()

fig.savefig("../Output/dpdd_t1000_angkor.png", dpi=300, bbox_inches="tight")
fig.savefig("../Output/dpdd_t1000_angkor.svg", bbox_inches="tight")
```

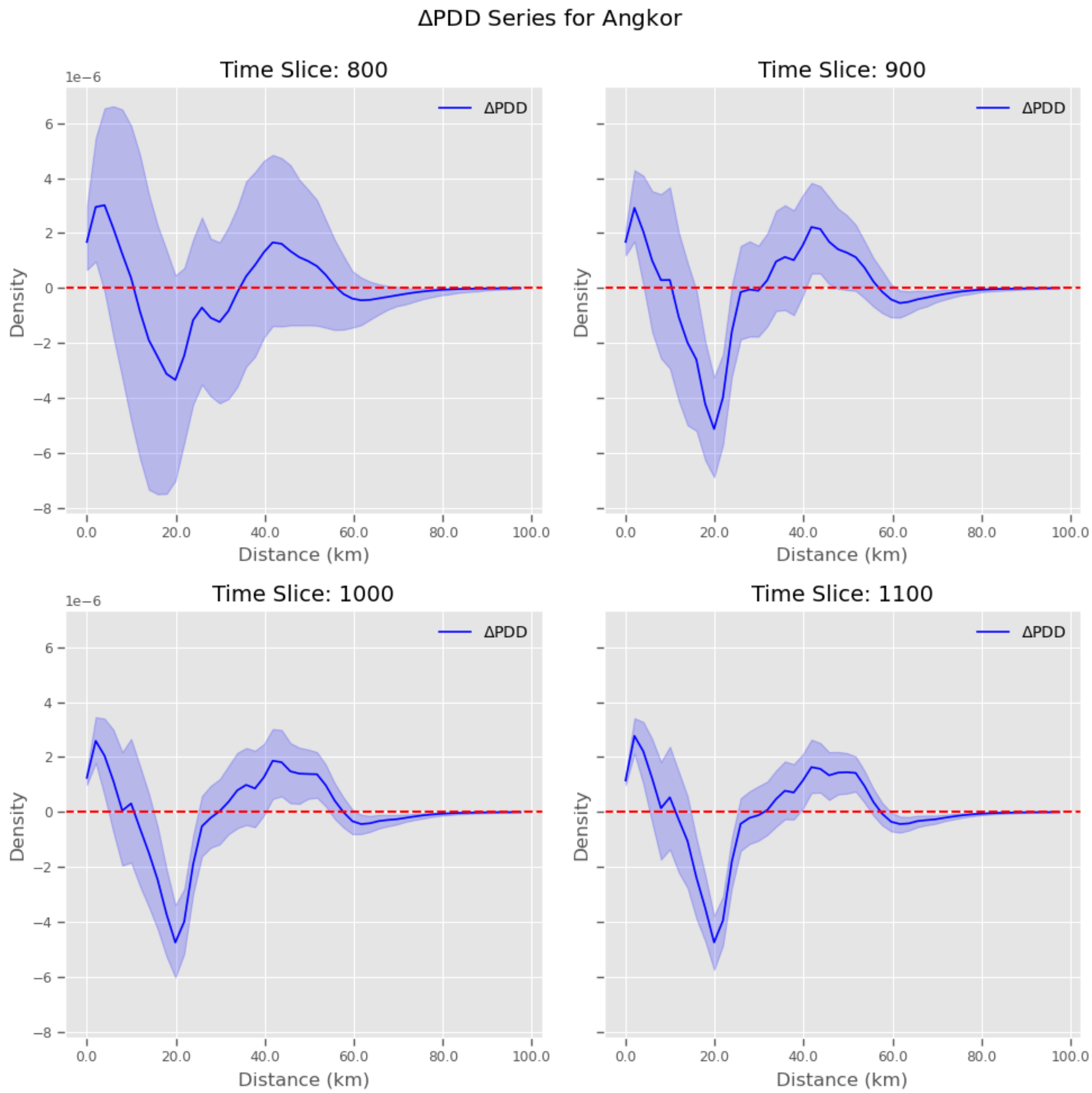


Series of Slices

Figure S8: Difference between Angkor PDD and BISE Null at 4 Time Slices

```
# List of time_slice_idx values
time_slice_indices = [0, 2, 4, 6]

# Create a figure and axes for subplots
# Create a 2x2 grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 10), sharey=True)  # 2 rows, 2
 ↪  columns

axes_flat = axes.flatten()

# Loop through each time_slice_idx and generate the plots
for idx, (ax, time_slice_idx) in enumerate(zip(axes_flat,
 ↪  time_slice_indices)):
    # Generate the plot for the current time_slice_idx
    fig, _ = plot_pdd(
        time_slices=time_slices,
        time_slice_idx=time_slice_idx,
```

```
        support=support_angkor,
        density_arrays=density_arrays,
        quantiles=[0.025, 0.975],
        density_names=["$\Delta$PDD"],
        colors=["blue"],
        ax=ax
    )

    # Add a horizontal line (optional)
    ax.axhline(y=0, color='red', linestyle='--', linewidth=1.5)

    # Add a title for each panel
    ax.set_title(f"Time Slice: {time_slices[time_slice_idx]}")
    ax.set_xlabel("Distance (m)")
    # Get current tick positions and convert labels to km
    x_ticks = ax.get_xticks()
    ax.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

    # Update axis label
    ax.set_xlabel("Distance (km)")

# Adjust layout and show the stitched plot
plt.tight_layout(rect=[0, 0, 1, 0.95])
fig.suptitle("$\Delta$PDD Series for Angkor")
plt.show()

fig.savefig("../Output/dpdd_series_angkor.png", dpi=300, bbox_inches="tight")
fig.savefig("../Output/dpdd_series_angkor.svg", bbox_inches="tight")
```



Hampshire County at Domesday

This section processes the Domesday data for Hampshire. The data were originally taken from an online repository and processed with another notebook in this repo (doomsday.ipynb). The processed data were then saved to the repo to reduce redundant processing for use in this notebook, but for full replication, you would need to run the cells in the other notebook before proceeding further here.

Data Wrangling

```
# data wrangling
doomsday_places = pd.read_csv('../Data/doomsday_places.csv')
doomsday_places = doomsday_places.dropna(subset=['easting', 'northing'])
doomsday_places[["County", "Hundred", "Vill", "easting", "northing"]]
```

	County	Hundred	Vill	easting	northing
0	WOR	‘Doddingtree'	Abberley	375000.0	267000.0
1	ESS	‘Winstree'	Abberton	599000.0	219000.0
2	WOR	Pershore	Abberton	399000.0	253000.0
3	DOR	‘Uggescombe'	Abbotsbury	357000.0	85000.0
4	DEV	Merton	Abbotsham	242000.0	126000.0
...	...	...	...	...	...
13453	STS	Offlow	Yoxall	414000.0	319000.0
13454	SUF	‘Blything'	Yoxford	639000.0	268000.0
13455	CHS	Ati's Cross	Ysceifiog	315000.0	371000.0
13456	DEV	North Tawton	Zeal Monachorum	271000.0	103000.0
13457	WIL	Mere	Zeals	378000.0	131000.0

Density Estimate

For simple comparison we estimate a density for Hampshire following the procedure used above for Angkor. We use a simple bounding convex hull to estimate the area (minimum bounding area containing all points) and then convert to km^2:

Includes removing two problematic points in the data with a likely incorrect county labels.

```
# isolating Hampshire for comparison with Angkor
counties = ['HAM']
doomsday_df = doomsday_places[doomsday_places['County'].isin(counties)]

# I know there is a probable county designation error for the following point
↪
# (observed in QGIS as an kind of spatial outlier surrounded by points  with
↪  a
# different designation and appears to be a duplicate point where the
↪  alternate
# one has the same county designation as the other surrounding points)
```

```python
# PlacesIdx of the mislabelled point is 10221 while the alternate is 10226
drop_idx = doomsday_df[doomsday_df['PlacesIdx'].isin([10221, 30086])].index
doomsday_df = doomsday_df.drop(drop_idx)
```

```python
# Build coordinate array
coords = doomsday_df[['easting', 'northing']].to_numpy()

# Compute convex hull
hull = ConvexHull(coords)

# In 2D, scipy's 'volume' is the polygon area
hull_area_m2 = hull.volume
hull_area_km2 = hull_area_m2 / 1_000_000

# Point count and density
n_points = len(coords)
density_per_km2 = n_points / hull_area_km2

print(f"Hampshire point count: {n_points}")
print(f"Convex hull area: {hull_area_m2:,.2f} m²")
print(f"Convex hull area: {hull_area_km2:,.2f} km²")
print(f"Point density: {density_per_km2:,.3f} points/km²")
```

```
Hampshire point count: 435
Convex hull area: 5,041,500,000.00 m²
Convex hull area: 5,041.50 km²
Point density: 0.086 points/km²
```

Create Points List

```python
doomsday_points = [
    clustering.Point(
        x=row['easting'],
        y=row['northing'],
        start_distribution = ddelta(1066),
        end_distribution = ddelta(1086)
    )
    for _, row in doomsday_df.iterrows()
```

```
]

# just double check the first ten look right
doomsday_points[:10]
```

```
[Point(x=456000.0, y=134000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=1
 Point(x=458000.0, y=88000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=10
 Point(x=459000.0, y=86000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=10
 Point(x=435000.0, y=86000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=10
 Point(x=456000.0, y=83000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=10
 Point(x=447000.0, y=117000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=1
 Point(x=427000.0, y=107000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=1
 Point(x=458000.0, y=133000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=1
 Point(x=471000.0, y=139000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=1
 Point(x=460000.0, y=98000.0, start_distribution=ddelta(d=1066), end_distribution=ddelta(d=10
```

Define Time Slices and Spatial Limits

```
# Define the time slices
start_time = 1066
end_time = 1086
time_interval = 5
time_slices_ham = np.arange(start_time, end_time, time_interval)

# Get a bounding box for use later and to extract sensible distance limits
x_min, y_min, x_max, y_max = get_box(doomsday_points)
max_distance = np.ceil(np.sqrt((x_max - x_min)**2 + (y_max - y_min)**2))
```

Figure S9: Spacetime Volume of Hampshire's Estates

```
# Custom styling parameters
style_params = {
    'start_mean_color': None,  # Do not plot start mean points
    'end_mean_color': None, # Do not plot end mean points
    'mean_point_size': 10,
    'cylinder_color': (0.3, 0.3, 0.3),  # Dark grey
    'ppf_limits': (0.05, 0.95),  # Use different ppf limits
    'shadow_color': (0.4, 0.4, 0.4),  # grey
```

```
    'shadow_size': 10,
    'time_slice_color': (0.5, 0.5, 0.5),  # Grey
    'time_slice_alpha': 0.3,
    'time_slice_point_color': (0, 0, 0),  # Black
}

# Plot the points using the chrono_plot function with
# custom styling and a time slice plane
ax_stv_doomsday, fig_stv_doomsday = chrono_plot(
    doomsday_points,
    style_params=style_params,
    time_slice=1076,
    title='Hamphsire'
)
ax_stv_doomsday.set_box_aspect(None, zoom=0.85)
plt.savefig("../Output/spacetime_volume_Hampshire.svg", bbox_inches='tight')
```

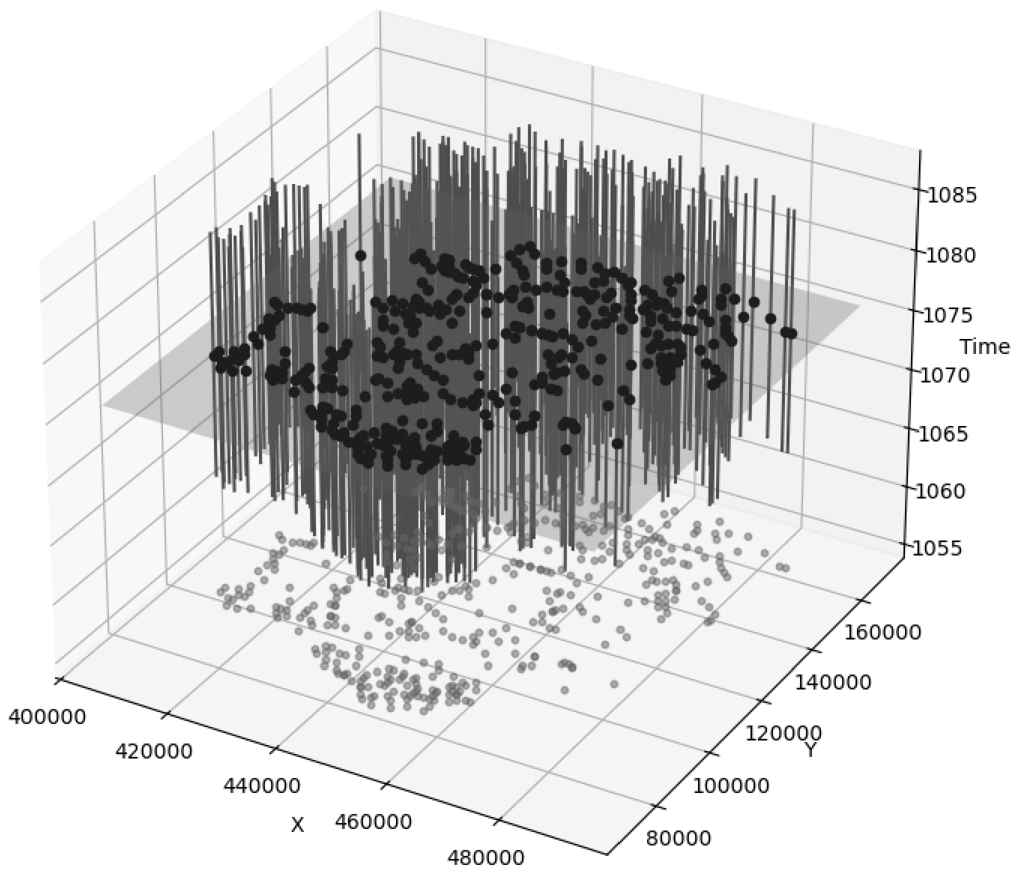


Figure S10: 2D Spatiotemporal Point Scatters of Angkor's Temples and Hampshire's Estates

```
# Style
scatter_style = {
    "point_color": (0, 0, 0),
    "point_size": 10,
}

width_mm = 90  # Example width in mm
```

```python
width_inch = width_mm / 25.4  # Convert mm to inches

# Create a figure with two subplots side by side (1 row, 2 columns)
fig, (ax_2d_angkor, ax_2d_hampshire) = plt.subplots(
    1,
    2,
    figsize = (2 * width_inch, 1.15 * width_inch)
)

y_delta = 0.18e6

# Plot for Angkor
ax_2d_angkor, fig_2d_angkor = chrono_plot2d(
    angkor_points,
    time = 1000,
    style_params = scatter_style,
    crs = "EPSG:32648",
    basemap_provider = ctx.providers.CartoDB.Voyager,
    ax = ax_2d_angkor,
)

ax_2d_angkor.set_title("Angkor 1000 CE", fontsize = 10)
ax_2d_angkor.set_xlabel("Easting", fontsize = 8)
ax_2d_angkor.set_ylabel("Northing", fontsize = 8)
ax_2d_angkor.tick_params(axis = 'both', labelsize = 8)

# Plot for Hampshire
ax_2d_hampshire, fig_2d_hampshire = chrono_plot2d(
    doomsday_points,
    time = 1076,
    style_params = scatter_style,
    crs = "EPSG:27700",
    basemap_provider = ctx.providers.CartoDB.Voyager,
    ax = ax_2d_hampshire,
)

ax_2d_hampshire.ticklabel_format(style = 'sci', scilimits = (0, 0))
ax_2d_hampshire.set_title("Hampshire 1076 CE", fontsize = 10)
ax_2d_hampshire.set_xlabel("Easting", fontsize = 8)
ax_2d_hampshire.set_ylabel("Northing", fontsize = 8)
ax_2d_hampshire.tick_params(axis = 'both', labelsize = 8)
```

```
inclusion_legend(
    ax = None,
    shared = True,
    fig = fig,
    alphas = [0.2, 0.5, 0.8, 1.0]
)

# Adjust layout
fig.tight_layout(rect = [0, 1, 0, 1])

# Save figure
plt.savefig(
    "../Output/combined_inclusion_scatter.svg",
    bbox_inches = "tight",
    dpi = 300
)
plt.savefig(
    "../Output/combined_inclusion_scatter.png",
    bbox_inches = "tight",
    dpi = 300)
```

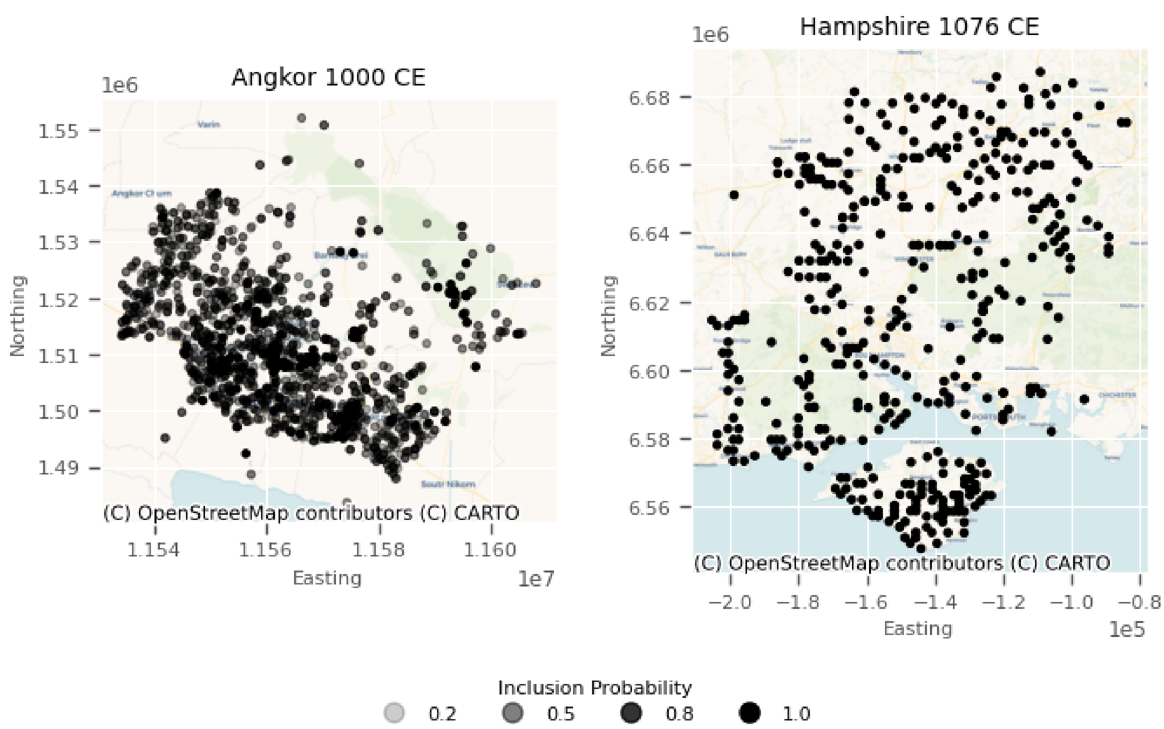
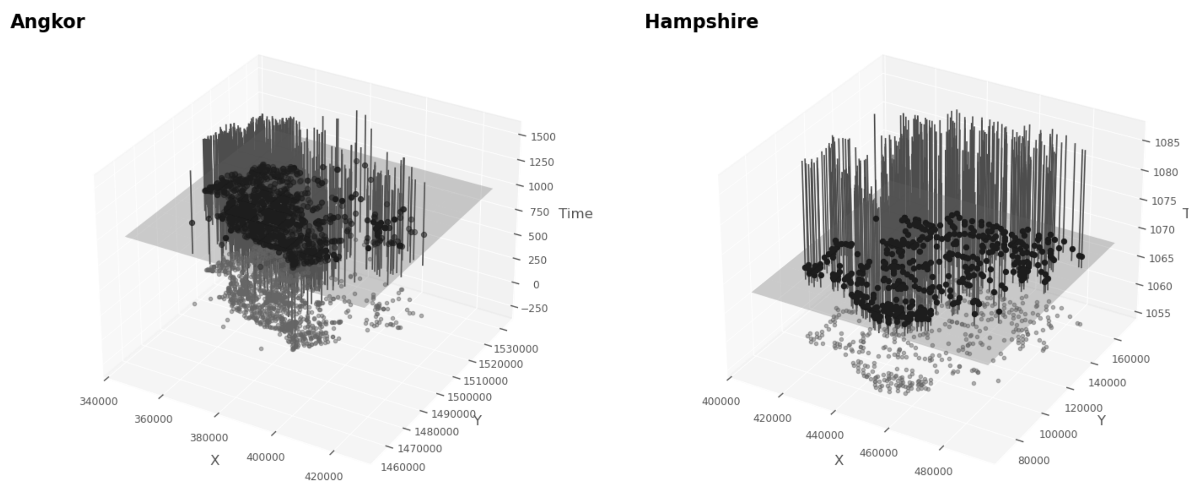


Figure S11: Spacetime Volumes for Angkor and Hampshire Combined

```
fig, axs = plt.subplots(
    1,
    2,
    figsize = (16, 6),
    subplot_kw = {"projection": "3d"}
)

# Plot both datasets into subplots
chrono_plot(
    points,
    ax = axs[0],
    style_params = style_params,
    time_slice = 1000
)
chrono_plot(
    doomsday_points,
    ax = axs[1],
    style_params = style_params,
    time_slice = 1068
)

# Add panel labels
fig.text(
    0.05,
    0.95,
    "Angkor",
    fontsize = 16,
    weight = "bold",
    transform = fig.transFigure
)
fig.text(
    0.52,
    0.95,
    "Hampshire",
    fontsize = 16,
    weight = "bold",
    transform = fig.transFigure
)

# Save combined figure
```

```
fig.tight_layout()

# Z-axis labels (use labelpad to bring them in from the edge)
axs[0].set_zlabel("Time", labelpad = 10)
axs[0].ticklabel_format(style = 'plain', axis = 'both')  # or 'y' or 'both'
axs[1].set_zlabel("Time", labelpad = 10)
axs[1].ticklabel_format(style = 'plain', axis = 'both')  # or 'y' or 'both'


fig.savefig(
    "../Output/spacetime_volume_combined.png",
    dpi = 300,
    bbox_inches = "tight",
    pad_inches = 1.0
)
fig.savefig(
    "../Output/spacetime_volume_combined.svg",
    bbox_inches = "tight",
    pad_inches = 1.0
)
```




```
# Run the Monte Carlo simulation to get an ensemble of probable
# lists of points included in each time slice.
num_iterations = 500
simulations_ham = clustering.mc_samples(
    doomsday_points,
    time_slices = time_slices_ham,
```

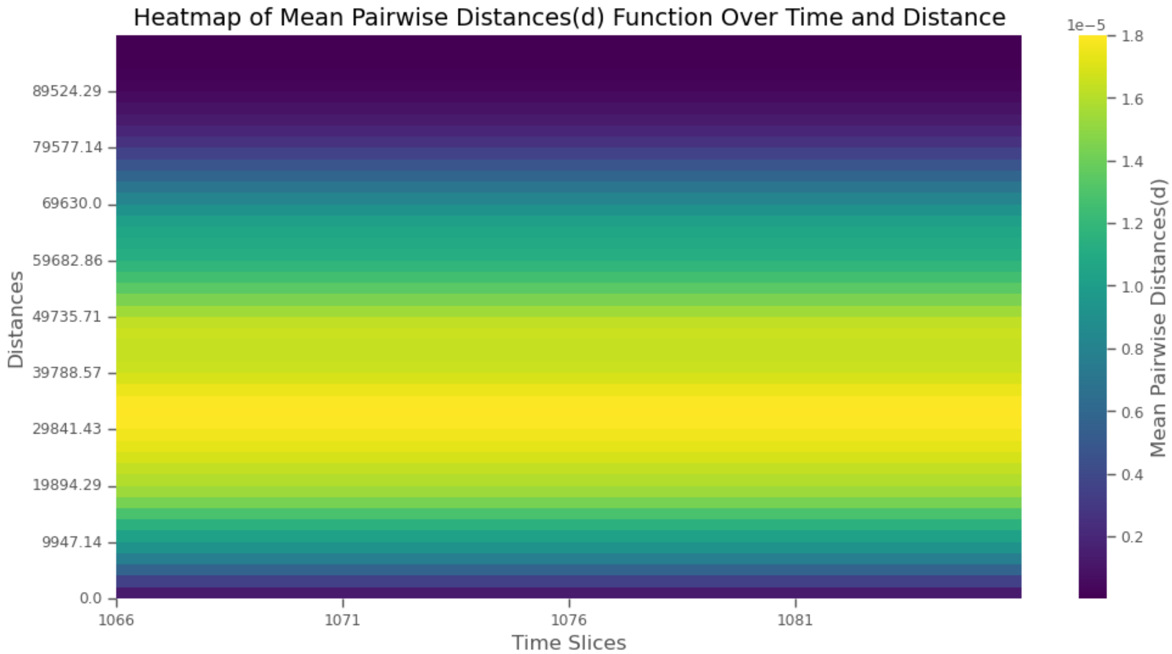
```
    num_iterations = num_iterations
)
```

Sampling and Pairwise Distance Density Estimation

```
# Get a bounding box for use later and to extract sensible distance limits
x_min, y_min, x_max, y_max = get_box(doomsday_points)
max_distance = np.ceil(np.sqrt((x_max - x_min)**2 + (y_max - y_min)**2))

# set consistent pairwise bandwidth (binning of distances)
# same as before with Angkor data
```

Figure S12: Heatmap of Pairwise Distance Density versus Time for Hampshire's Estates

```
# Produce pairwise distances to explore clustering structure
pairwise_density_ham, support_ham = clustering.temporal_pairwise(
    simulations_ham,
    time_slices_ham,
    bw=pair_bw,
    use_kde = use_kde,
    kde_sample_n = kde_sample_n,
    max_distance = max_distance,
    kde_custom = kde_custom
)

# Visualize clustering with heatmap
fig, ax = clustering_heatmap(
    pairwise_density_ham,
    support_ham,
    time_slices_ham,
    result_type = 'Pairwise Distances'
)

ax.set_ylabel("Distance (m)")
ax.set_xlabel("Time Slices (year CE)")

plt.savefig("../Output/pdd_hm_hampshire.svg", bbox_inches='tight')
plt.savefig("../Output/pdd_hm_hampshire.png", bbox_inches='tight')
```

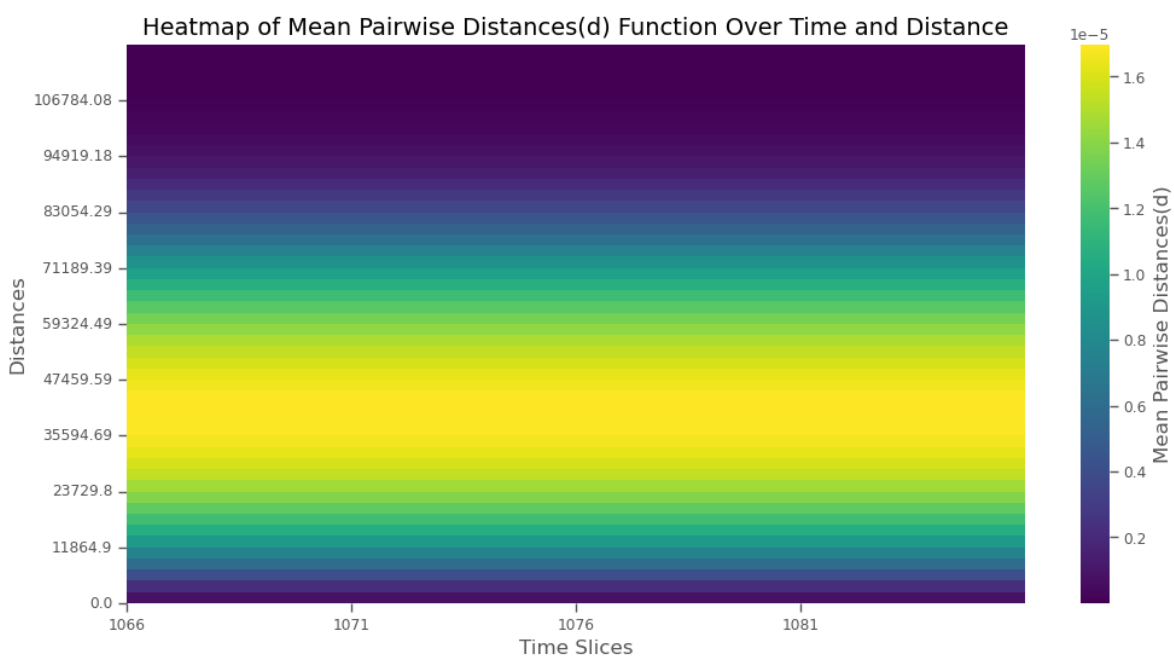


Complete Spatial Randomness

Figure S13: Heatmap of PDD versus Time of CSR for Hampshire

```
# Get MC iterations for incorporating chronological uncertainty and CSR
csr_simulations_ham = clustering.mc_samples(
    doomsday_points,
    time_slices = time_slices_ham,
    num_iterations = num_iterations,
    null_model = clustering.csr_sample,
    x_min = x_min,
    x_max = x_max,
    y_min = y_min,
    y_max = y_max
)

# Calulate the pairwise distances for the CSR sample
csr_pairwise_density_ham, csr_support_ham = clustering.temporal_pairwise(
    csr_simulations_ham,
    time_slices_ham,
    bw = pair_bw,
```

```
    use_kde = use_kde,
    kde_sample_n = kde_sample_n,
    max_distance = max_distance,
    kde_custom = kde_custom
)

# Visualize clustering with heatmap
fig, ax = clustering_heatmap(
    csr_pairwise_density_ham,
    csr_support_ham,
    time_slices_ham,
    result_type = 'Pairwise Distances'
)

ax.set_ylabel("Distance (m)")
ax.set_xlabel("Time Slices (year CE)")
```



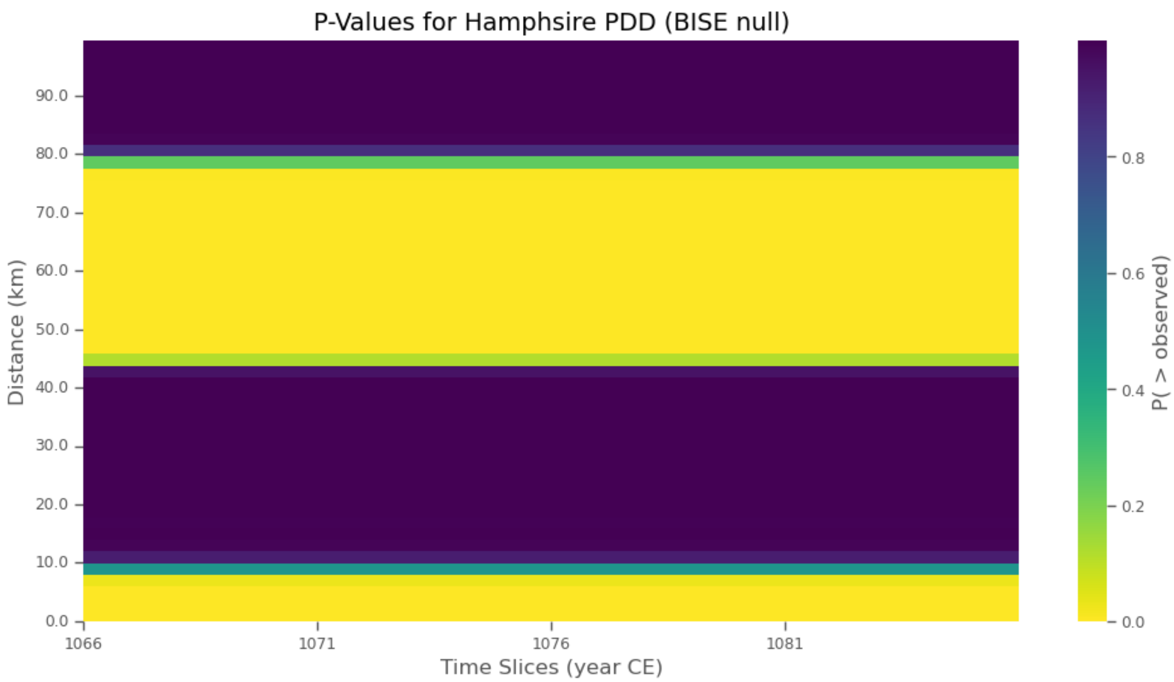
Baseline-Informed Spatial Expectation

Figure S14: Heatmap of PDD Statistical Significance compred to BISE for Hampshire

```
# Get MC iterations for incorporating chronological uncertainty with BISE
bise_simulations_ham = clustering.mc_samples(
    doomsday_points,
    time_slices_ham,
    num_iterations=num_iterations,
    null_model=clustering.bise
)

# Calulate the pairwise distances for the LISE sample
bise_pairwise_density_ham, bise_support_ham = clustering.temporal_pairwise(
    bise_simulations_ham,
    time_slices_ham,
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n=kde_sample_n,
    max_distance = max_distance,
    kde_custom=kde_custom
)

# Calculate the p-values for density differences between
# the observed points and the simulated CSR baseline per
# distance and temporal slice
p_diff_array_bise_ham, diff_array_bise_ham = clustering.p_diff(
    pairwise_density_ham,
    bise_pairwise_density_ham
)
```

```
# Plot the heatmap of probabilities
fig, ax = pdiff_heatmap(
    p_diff_array_bise_ham,
    time_slices_ham,
    bise_support_ham
)

# Custom ticks and labels here
tick_labels_km = np.arange(0, bise_support_ham.max() / 1000, 10)
tick_labels_m = tick_labels_km * 1000
tick_positions = np.interp(
    tick_labels_m,
    bise_support_ham,
    np.arange(len(bise_support_ham))
```

```
)

ax.set_yticks(tick_positions)
ax.set_yticklabels(np.round(tick_labels_km, 1))
ax.set_ylabel("Distance (km)")

ax.set_xlabel("Time Slices (year CE)")
ax.set_title("P-Values for Hamphsire PDD (BISE null)")

plt.savefig("../Output/dpdd_hm_ham.svg", bbox_inches='tight')
plt.savefig("../Output/dpdd_hm_ham.png", bbox_inches='tight')
```



One Time Slice

Figure S15: PDD of Hampshire Estates compared to CSR and BISE Null Models at 1066 CE

```
time_slice_idx = np.where(time_slices_ham == 1066)[0][0]

# List of density arrays
```

```python
density_arrays = [
    pairwise_density_ham,
    csr_pairwise_density_ham,
    bise_pairwise_density_ham]

# Generate the plot and get the figure and axis objects
fig, ax = plot_pdd(
    time_slices=time_slices_ham,
    time_slice_idx=time_slice_idx,
    support=support_ham,
    density_arrays=density_arrays,
    quantiles=[0.025, 0.975],
    density_names=["Empirical", "CSR", "BISE"],
    colors=["blue", "orange", "green"]
)

ax.set_title("PDD Hampshire 1066 CE")

# Get current tick positions and convert labels to km
x_ticks = ax.get_xticks()
ax.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax.set_xlabel("Distance (km)")

# Show the plot
plt.show()

fig.savefig(
    "../Output/pdd_null_hampshire.png",
    dpi = 300,
    bbox_inches = "tight"
)
fig.savefig(
    "../Output/pdd_null_hampshire.svg",
    bbox_inches = "tight"
)
```

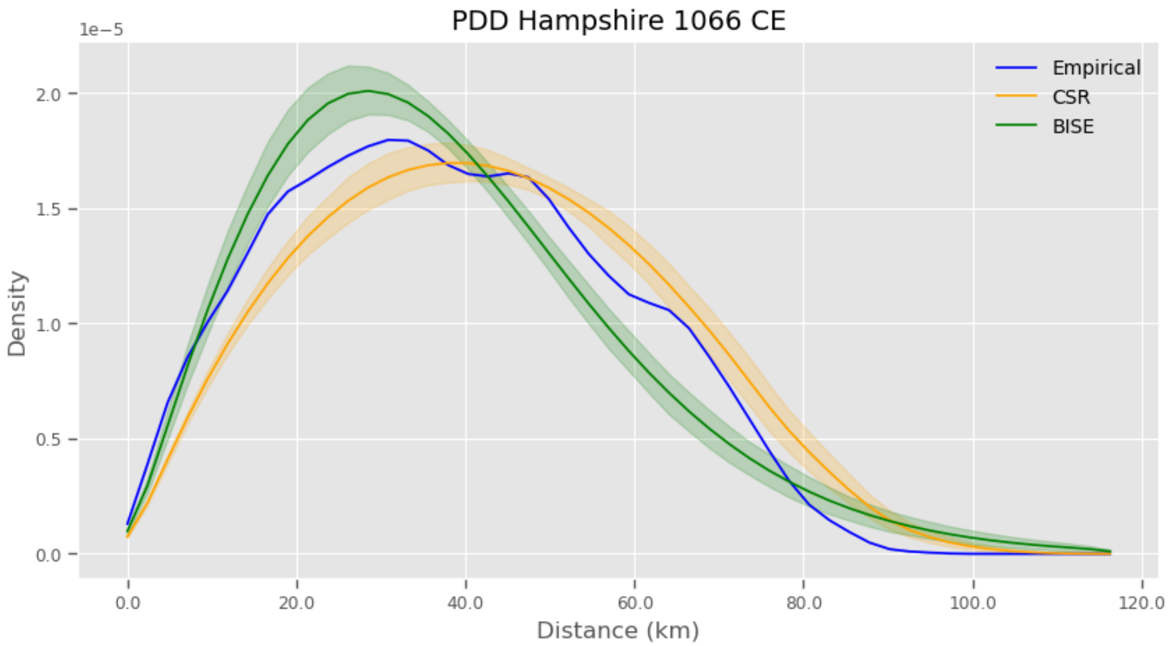


Figure S16: Difference between PDD and BISE Null Model for Hampshire's Estates at 1066 CE

```
# List of density arrays
density_arrays = [diff_array_bise_ham]

# Generate the plot and get the figure and axis objects
fig, ax = plot_pdd(
    time_slices=time_slices_ham,
    time_slice_idx=time_slice_idx,
    support=support_ham,
    density_arrays=density_arrays,
    quantiles=[0.025, 0.975],
    density_names=["$\Delta$PDD"],
    colors=["blue"]
)

# Add a horizontal line at y=0
ax.axhline(y=0, color='red', linestyle='--', linewidth=1.5)

ax.set_title("$\Delta$PDD Hampshire 1066 CE")
#ax.set_xlabel("Distance (m)")
```

```
# Get current tick positions and convert labels to km
x_ticks = ax.get_xticks()
ax.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax.set_xlabel("Distance (km)")

# Show the plot
plt.show()
```

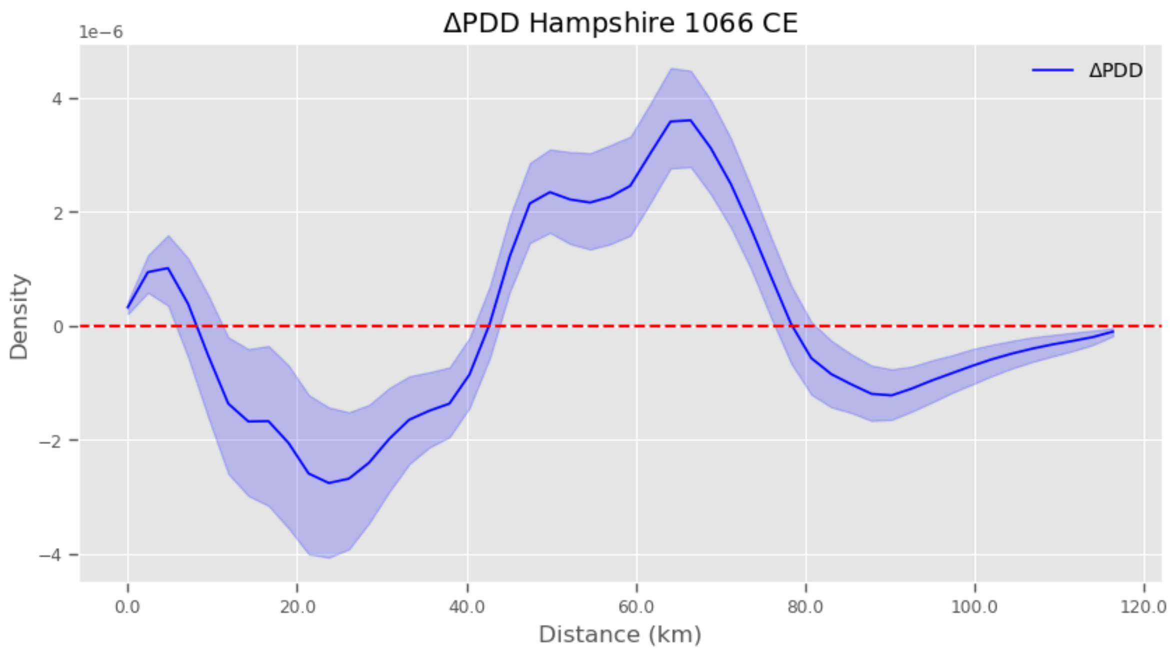


Figure S17: △ PDDs of Angkor and Hampshire Side by Side

```
# Create a figure with two side-by-side subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5), sharex=False,
↪   sharey=True)

# First plot, Angkor
time_slice_idx = np.where(time_slices == 1000)[0][0]
plot_pdd(
    time_slices = time_slices,
```

```
    time_slice_idx = time_slice_idx,
    support = support_angkor,
    density_arrays = [diff_array_bise_angkor],
    quantiles = [0.025, 0.975],
    density_names = ["$\Delta$PDD Angkor"],
    colors = ["blue"],
    ax=ax1
)
ax1.axhline(y=0, color='red', linestyle='--', linewidth=1.5)
ax1.set_title("$\Delta$PDD Angkor")
#ax1.set_xlabel("Distance (m)")

# Get current tick positions and convert labels to km
x_ticks = ax1.get_xticks()
ax1.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax1.set_xlabel("Distance (km)")

# Second plot, Hampshire
time_slice_idx = np.where(time_slices_ham == 1066)[0][0]
plot_pdd(
    time_slices = time_slices_ham,
    time_slice_idx = time_slice_idx,
    support = support_ham,
    density_arrays = [diff_array_bise_ham],
    quantiles = [0.025, 0.975],
    density_names = ["$\Delta$PDD Hampshire"],
    colors = ["green"],
    ax=ax2
)
ax2.axhline(y=0, color='red', linestyle='--', linewidth=1.5)
ax2.set_title("$\Delta$PDD Hampshire")
#ax2.set_xlabel("Distance (m)")

# Get current tick positions and convert labels to km
x_ticks = ax2.get_xticks()
ax2.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax2.set_xlabel("Distance (km)")
```

```
# Adjust layout and show the combined plot
plt.tight_layout()
plt.show()

fig.savefig(
    "../Output/dpdd_compared.png",
    dpi = 300,
    bbox_inches = "tight"
)
fig.savefig(
    "../Output/dpdd_compared.svg",
    bbox_inches = "tight"
)
```

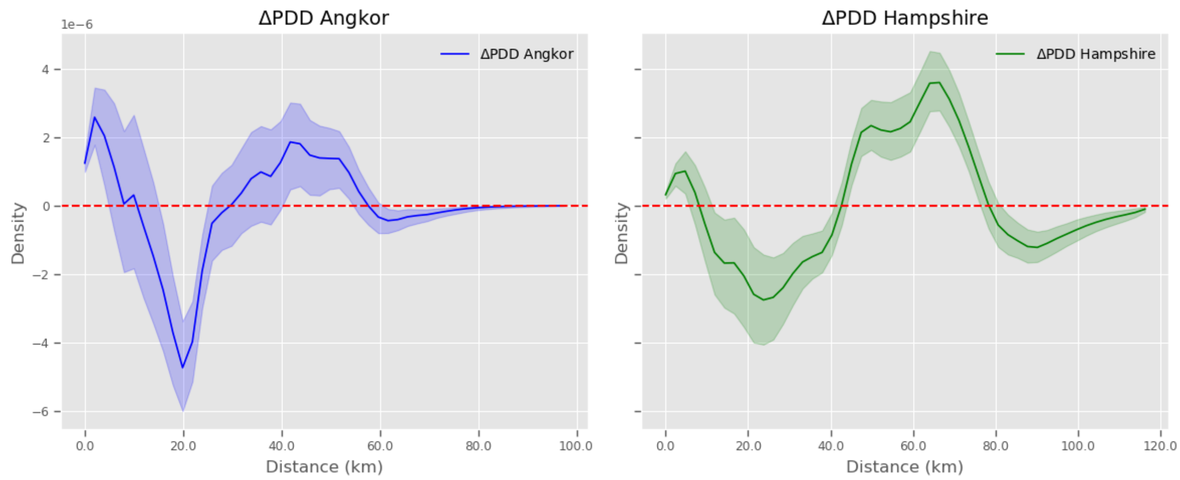
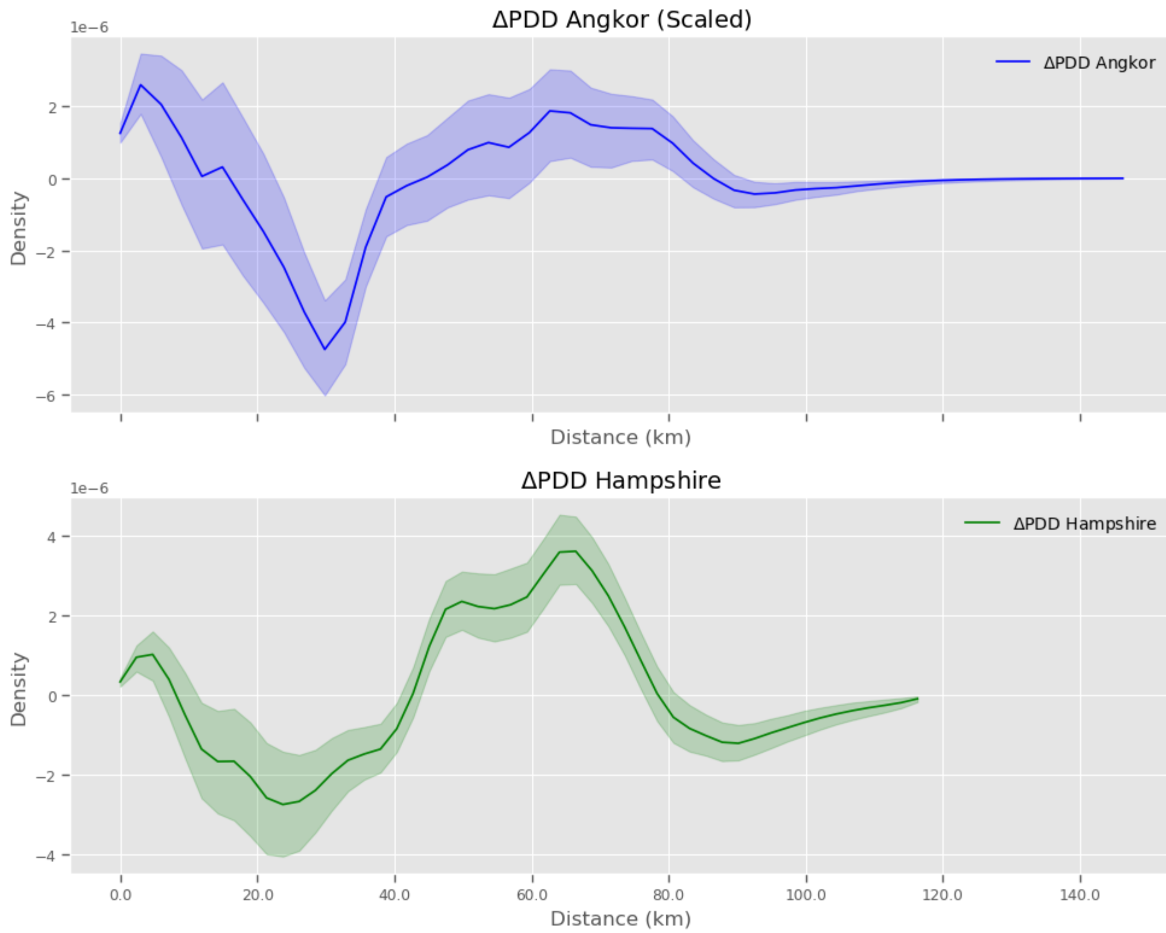


Figure S18: △ PDDs compared with Angkor's PDD Scaled

```
# Scaling ratio for Angkor x-axis
scaling_ratio = 1.5
scaled_support_angkor = np.array(support_angkor) * scaling_ratio

# Create a vertically stacked figure with shared x-axis
fig, (ax1, ax2) = plt.subplots(
    2, 1,
    figsize=(10, 8),
    sharex=True,
    gridspec_kw={'height_ratios': [1, 1]}
```

```
)

# First plot: Angkor (scaled x-axis)
time_slice_idx_angkor = np.where(time_slices == 1000)[0][0]
plot_pdd(
    time_slices=time_slices,
    time_slice_idx=time_slice_idx_angkor,
    support=scaled_support_angkor,
    density_arrays=[diff_array_bise_angkor],
    quantiles=[0.025, 0.975],
    density_names=["$\Delta$PDD Angkor"],
    colors=["blue"],
    ax=ax1
)
ax1.set_title("$\Delta$PDD Angkor (Scaled)")
ax1.tick_params(labelbottom=False)
#ax1.set_xlabel("Distance (m)")

# Get current tick positions and convert labels to km
x_ticks = ax1.get_xticks()
ax1.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax1.set_xlabel("Distance (km)")

# Second plot: Hampshire
time_slice_idx_ham = np.where(time_slices_ham == 1066)[0][0]
plot_pdd(
    time_slices = time_slices_ham,
    time_slice_idx = time_slice_idx_ham,
    support = support_ham,
    density_arrays = [diff_array_bise_ham],
    quantiles = [0.025, 0.975],
    density_names = ["$\Delta$PDD Hampshire"],
    colors = ["green"],
    ax=ax2
)
ax2.set_title("$\Delta$PDD Hampshire")
#ax2.set_xlabel("Distance (m)")

# Get current tick positions and convert labels to km
x_ticks = ax2.get_xticks()
```

```python
ax2.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax2.set_xlabel("Distance (km)")

# Final layout tweaks
plt.tight_layout()
plt.show()

fig.savefig(
    "../Output/dpdd_scaled.png",
    dpi=300,
    bbox_inches="tight"
)
fig.savefig(
    "../Output/dpdd_scaled.svg",
    bbox_inches="tight"
)
```



First Peaks

```
def find_first_peak(pdd_slice, support):
    """
    Finds the first peak in a PDD slice.

    Parameters:
    -----------
    pdd_slice : np.ndarray
        A 1D array of PDD values for a single realization.
    support : np.ndarray
        Array of distance values (x-axis).
```

```
    Returns:
    --------
    float
        Distance (x-coordinate) of the first peak.
    """
    # Find all peaks in the PDD slice
    peaks, _ = find_peaks(pdd_slice)

    # If peaks exist, return the first one
    if len(peaks) > 0:
        return support[peaks[0]]

    # If no peaks are found, return NaN
    return np.nan

def find_all_first_peaks(diff_array, support, time_slice_idx):
    """
    Finds the first peak for all realizations in a PDD difference array and
 ↪  returns
    both the peak locations and their corresponding densities.

    Parameters:
    -----------
    diff_array : np.ndarray
        3D array of PDD difference values (distances x time_slices x
 ↪  realizations).
    support : np.ndarray
        Array of distance values (x-axis).
    time_slice_idx : int
        Index of the time slice to analyze.

    Returns:
    --------
    peaks : list
        List of first peak locations for all realizations.
    densities : list
        List of density values at the first peak for all realizations.
    """
    peaks = []
    densities = []
    num_realizations = diff_array.shape[2]
```

```
    for realization_idx in range(num_realizations):
        # Extract the PDD slice for the current realization
        pdd_slice = diff_array[:, time_slice_idx, realization_idx]

        # Find the first peak location
        peak_location = find_first_peak(pdd_slice, support)

        # if no peak, just return nan
        if np.isnan(peak_location):
            warnings.warn("No peak found.", UserWarning)
            peaks.append(np.nan)
            densities.append(np.nan)
        else:
            # Get the density value at the peak
            peak_density = pdd_slice[support == peak_location][0]

            # Append results
            peaks.append(peak_location)
            densities.append(peak_density)

    return np.array(peaks), np.array(densities)
```

Set Common Parameters

```
num_iterations = 500
use_kde = True
pair_bw = None
kde_sample_n = 100
kde_custom=cuml_kde
max_distance = 15000
```

Angkor First Peak

```
time_slice = 1100

# Run the Monte Carlo simulation to get an ensemble of probable
# lists of points included in each time slice.
```

```
simulations = clustering.mc_samples(
    points,
    time_slices=[time_slice],
    num_iterations=num_iterations
)

# Produce pairwise distances to explore clustering structure
pairwise_density_angkor, support_angkor = clustering.temporal_pairwise(
    simulations,
    [time_slice],
    bw=pair_bw,
    use_kde=use_kde,
    kde_sample_n=kde_sample_n,
    max_distance=max_distance,
    kde_custom=kde_custom
)

# Get MC iterations for incorporating chronological uncertainty with BISE
bise_simulations = clustering.mc_samples(
    points,
    [time_slice],
    num_iterations=num_iterations,
    null_model=clustering.bise
)

# Calulate the pairwise distances for the LISE sample
bise_pairwise_density_angkor, bise_support_angkor =
 ↪  clustering.temporal_pairwise(
    bise_simulations,
    [time_slice],
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n=kde_sample_n,
    max_distance = max_distance,
    kde_custom=kde_custom
)

# Calculate the p-values for density differences between the observed points
 ↪  and
# the simulated CSR baseline per distance and temporal slice
p_diff_array_angkor, diff_array_angkor = clustering.p_diff(
    pairwise_density_angkor,
```

```
    bise_pairwise_density_angkor
)
```

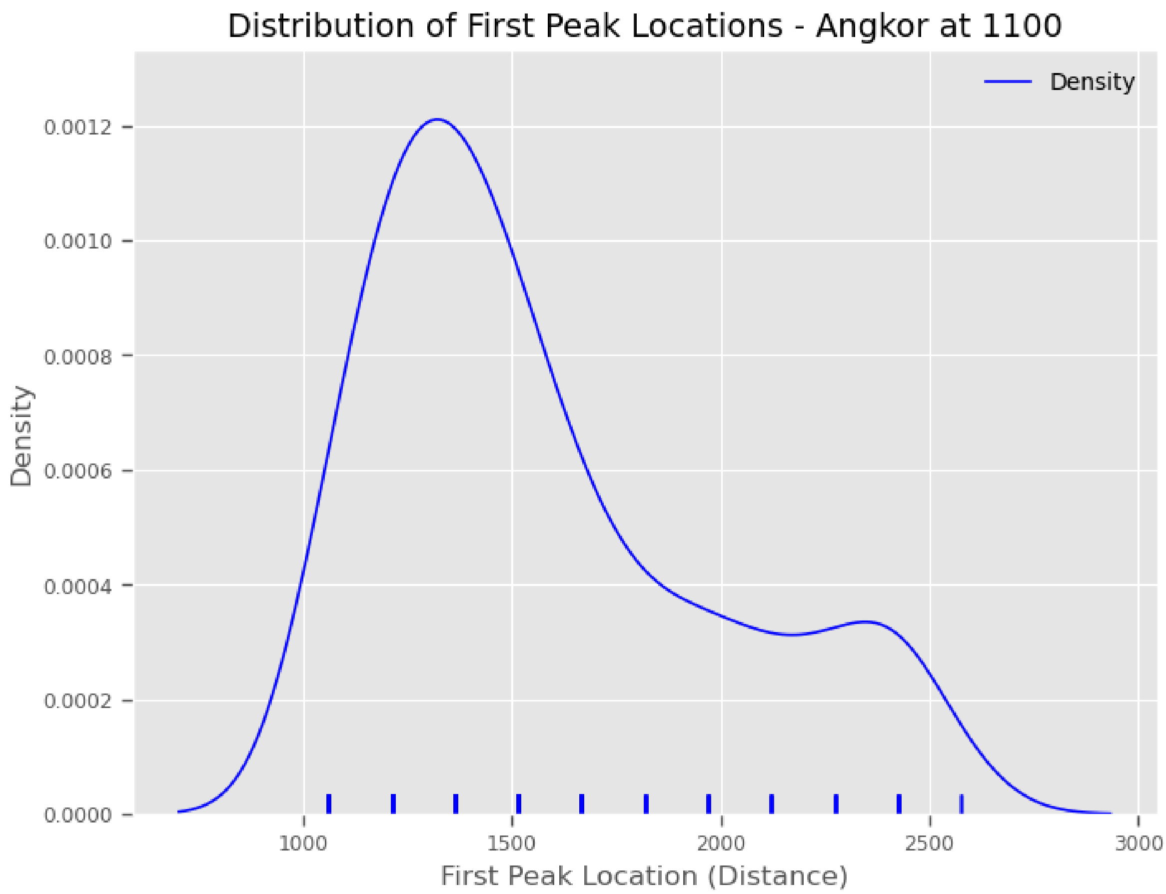
```
p_pdd_peaks_angkor, _ = find_all_first_peaks(
    diff_array_angkor,
    support_angkor,
    0
)

# Convert to a Pandas DataFrame and use describe()
summary_stats = pd.DataFrame(p_pdd_peaks_angkor,
 ↪  columns=["Values"]).describe()

# Display the summary statistics
summary_stats
```

	Values
count	500.000000
mean	1575.757576
std	413.135720
min	1060.606061
25%	1212.121212
50%	1515.151515
75%	1818.181818
max	2575.757576

Figure S19: Distribution of First Peak Locations for Angkor at 1000 CE

```
# Assuming `peaks` is your data
# Plot density and rug plot
plt.figure(figsize=(8, 6))
sns.kdeplot(p_pdd_peaks_angkor, color='blue', label="Density")
sns.rugplot(p_pdd_peaks_angkor, color='blue', alpha=0.5)

# Add labels and title
plt.xlabel("First Peak Location (Distance m)")
plt.ylabel("Density")
plt.title(f"Distribution of First Peak Locations - Angkor at {time_slice}")
plt.legend()
```

```
# Show the plot
plt.show()
```



Hampshire First Peak

```
time_slice = 1066

# Run the Monte Carlo simulation to get an ensemble of probable
# lists of points included in each time slice.
simulations = clustering.mc_samples(
    doomsday_points,
    time_slices=[time_slice],
```

```
    num_iterations=num_iterations
)

# Produce pairwise distances to explore clustering structure
pairwise_density_hampshire, support_hampshire = clustering.temporal_pairwise(
    simulations,
    [time_slice],
    bw=pair_bw,
    use_kde=use_kde,
    kde_sample_n=kde_sample_n,
    max_distance=max_distance,
    kde_custom=kde_custom
)

# Get MC iterations for incorporating chronological uncertainty with BISE
bise_simulations = clustering.mc_samples(
    doomsday_points,
    [time_slice],
    num_iterations=num_iterations,
    null_model=clustering.bise
)

# Calulate the pairwise distances for the BISE sample
bise_pairwise_density_hampshire, bise_support_hampshire =
 ↪  clustering.temporal_pairwise(
    bise_simulations,
    [time_slice],
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n=kde_sample_n,
    max_distance = max_distance,
    kde_custom=kde_custom
)

# Calculate the p-values for density differences between the observed points
# and the simulated CSR baseline per distance and temporal slice
p_diff_array_hampshire, diff_array_hampshire = clustering.p_diff(
    pairwise_density_hampshire,
    bise_pairwise_density_hampshire
)
```

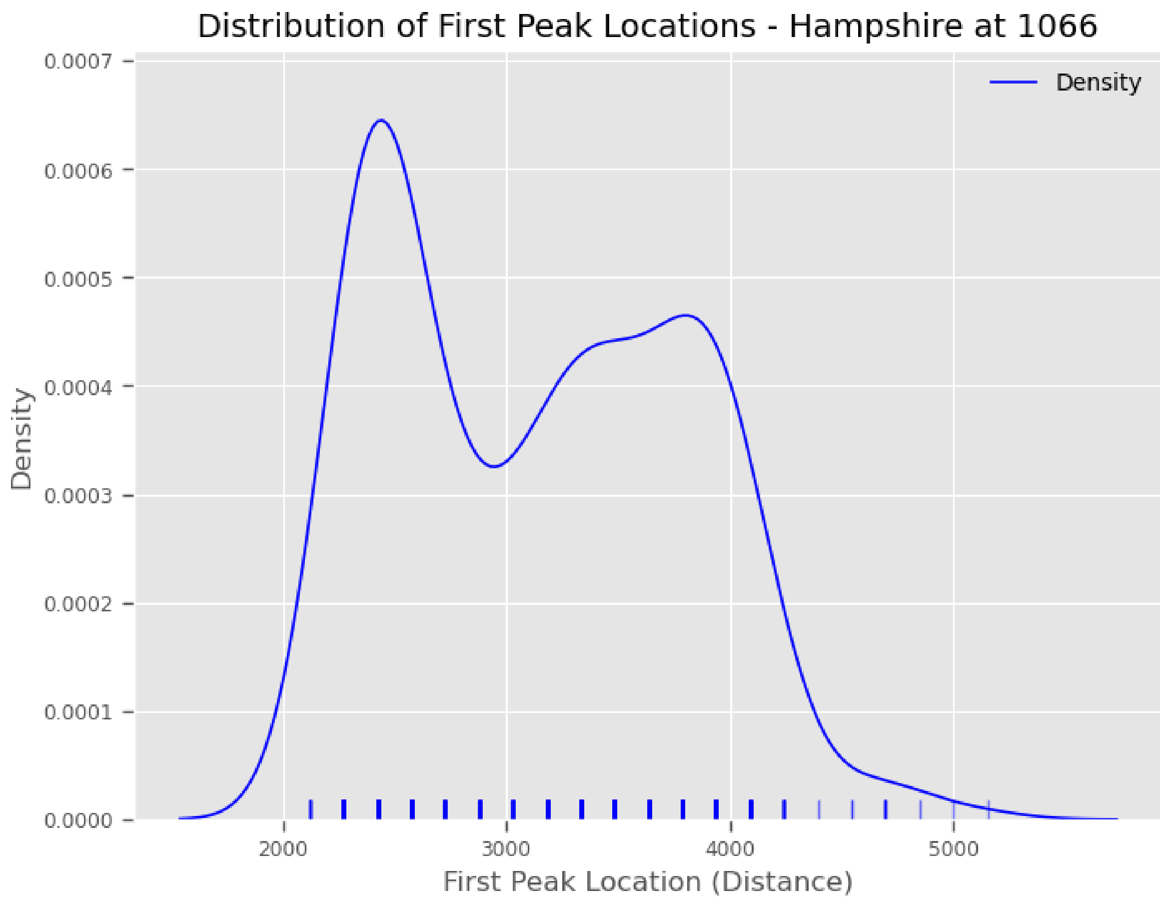
```python
p_pdd_peaks_hampshire, _ = find_all_first_peaks(
    diff_array_hampshire,
    support_hampshire,
    0
)

# Convert to a Pandas DataFrame and use describe()
summary_stats = pd.DataFrame(
    p_pdd_peaks_hampshire,
    columns = ["Values"]
).describe()

# Display the summary statistics
summary_stats
```

	Values
count	500.000000
mean	3146.060606
std	669.473271
min	2121.212121
25%	2424.242424
50%	3181.818182
75%	3787.878788
max	5151.515152

Figure S20: Distribution of First Peak Locations for Hampshire at 1066 CE

```python
# Assuming `peaks` is your data
# Plot density and rug plot
plt.figure(figsize=(8, 6))
sns.kdeplot(p_pdd_peaks_hampshire, color='blue', label="Density")
sns.rugplot(p_pdd_peaks_hampshire, color='blue', alpha=0.5)

# Add labels and title
plt.xlabel("First Peak Location (Distance m)")
plt.ylabel("Density")
plt.title(f"Distribution of First Peak Locations - Hampshire at
↪ {time_slice}")
plt.legend()
```

```
# Show the plot
plt.show()
```

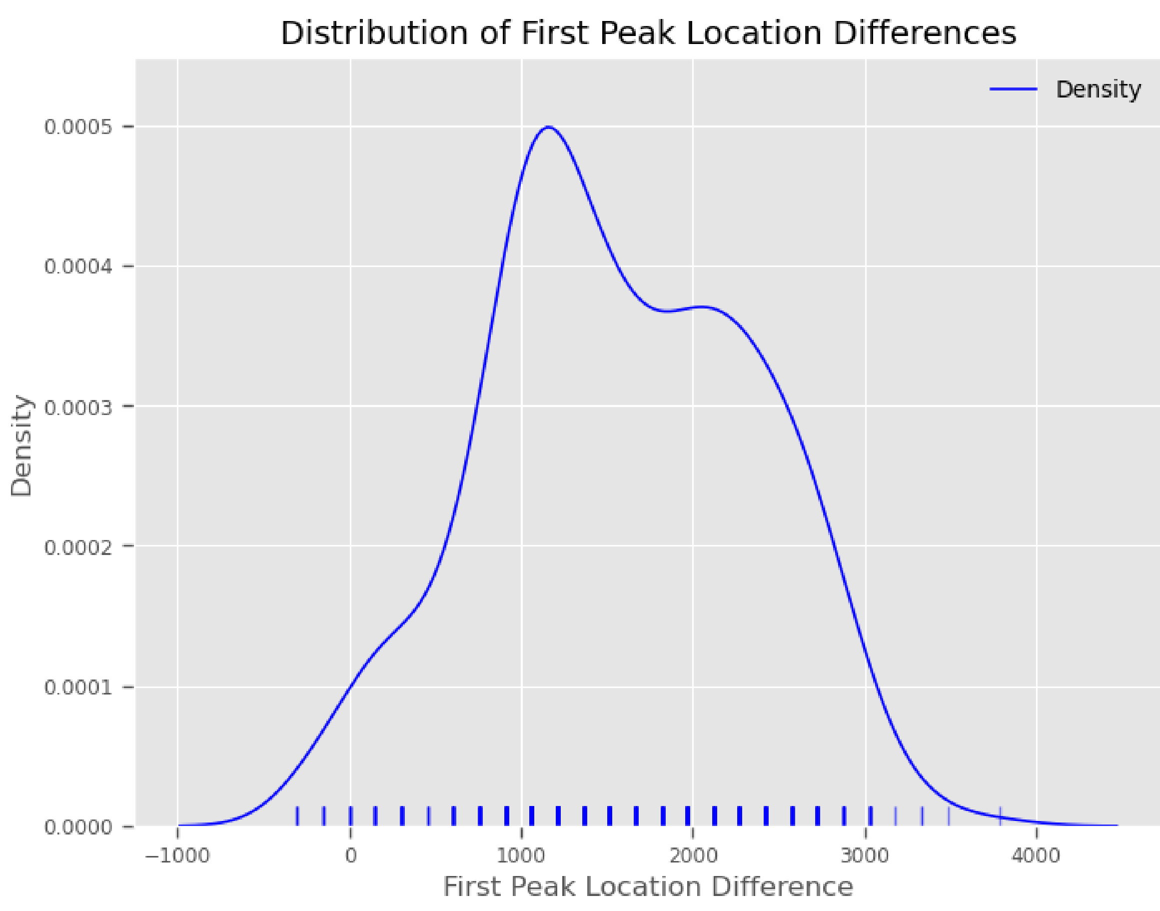


Difference Distribution

Figure S21: Distribution of First Peak Location Differences between Angkor and Hampshire

```
# Assuming `peaks` is your data
# Plot density and rug plot
plt.figure(figsize=(8, 6))
sns.kdeplot(
    p_pdd_peaks_hampshire - p_pdd_peaks_angkor,
```

```
    color = 'blue',
    label = "Density")
sns.rugplot(
    p_pdd_peaks_hampshire - p_pdd_peaks_angkor,
    color = 'blue',
    alpha = 0.5)

# Add labels and title
plt.xlabel("First Peak Location Difference (m)")
plt.ylabel("Density")
plt.title("Distribution of First Peak Location Differences")
plt.legend()

# Show the plot
plt.show()
```



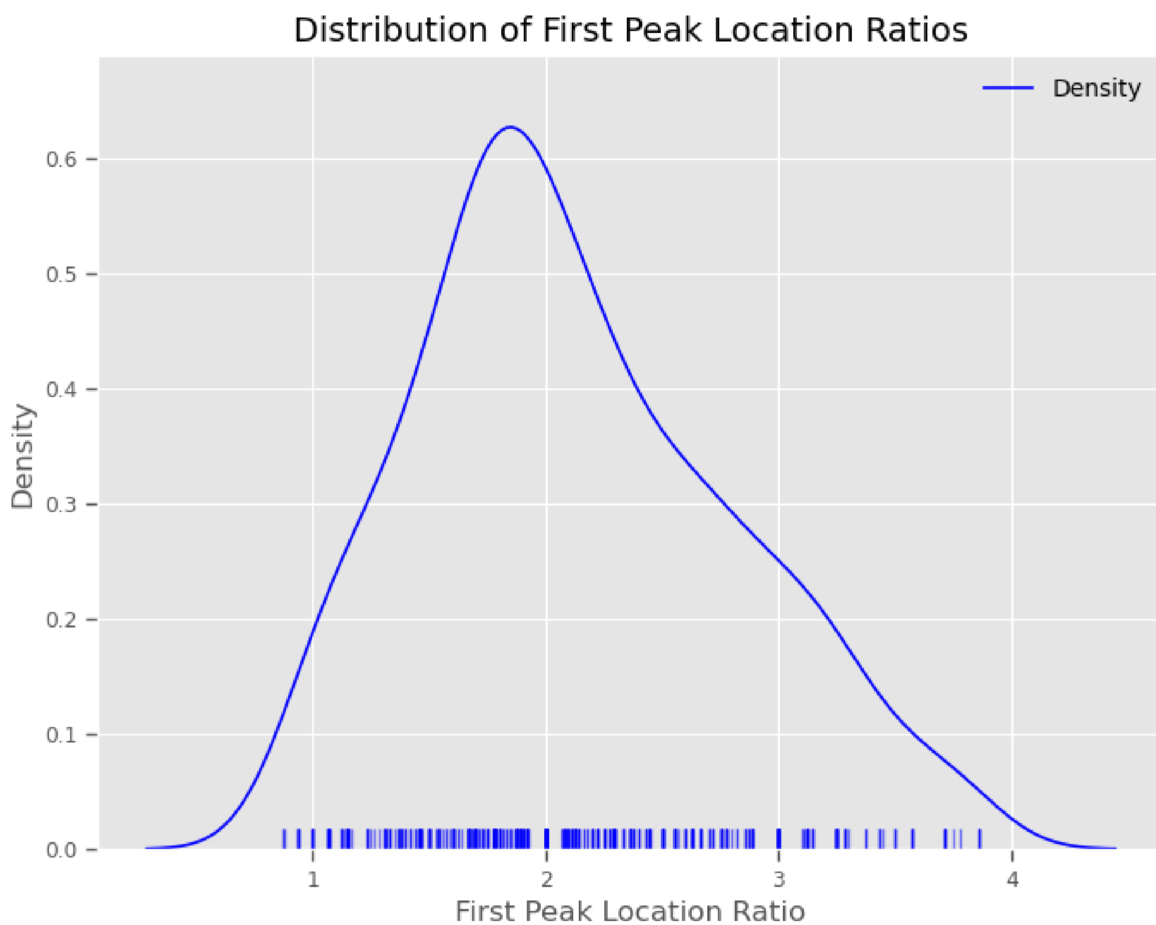
```
# Convert to a Pandas DataFrame and use describe()
summary_stats = pd.DataFrame(
    p_pdd_peaks_hampshire / p_pdd_peaks_angkor,
    columns = ["Values"]
).describe()

# Display the summary statistics
summary_stats
```

	Values
count	500.000000
mean	2.123158
std	0.678257
min	0.875000
25%	1.666667
50%	2.000000
75%	2.600000
max	3.857143

Figure S22: Distribution of First Peal Location Ratios between Angkor and Hampshire

```
# Assuming `peaks` is your data
# Plot density and rug plot
plt.figure(figsize=(8, 6))
sns.kdeplot(
    p_pdd_peaks_hampshire / p_pdd_peaks_angkor,
    color = 'blue',
    label = "Density"
)
sns.rugplot(
    p_pdd_peaks_hampshire / p_pdd_peaks_angkor,
    color = 'blue',
    alpha = 0.5
)

# Add labels and title
plt.xlabel("First Peak Location Ratio")
plt.ylabel("Density")
```

```
plt.title("Distribution of First Peak Location Ratios")
plt.legend()

# Show the plot
plt.show()
```



Extended Analyses

Domesday

To test the robsutness of our findings to county selection, we re-ran the core analysis for each of the counties listed in the Domesday archive. The results are stored in a series of plots that will be in the Output folder of the assocaited repo and can be reproduced with the following code.

```
# get unique list of counties
counties = doomsday_places['County'].unique()

# Ensure the Output directory exists
output_dir = "../Output"
os.makedirs(output_dir, exist_ok=True)

# loop over it
for j in tqdm(counties, desc="Processing Counties"):
    # isolate j
    doomsday_df = doomsday_places[doomsday_places['County'] == j]

    # create list of points
    doomsday_points = [
    clustering.Point(
        x=row['easting'],
        y=row['northing'],
        start_distribution = ddelta(1066),
        end_distribution = ddelta(1086)
    )
    for _, row in doomsday_df.iterrows()
    ]

    # Define the time slices
    start_time = 1066
    end_time = 1086
    time_interval = 5
    time_slices = np.arange(start_time, end_time, time_interval)
    time_slices

    # Get a bounding box for use later and to extract sensible distance
     ↪  limits
    x_min, y_min, x_max, y_max = get_box(doomsday_points)
    max_distance = np.ceil(np.sqrt((x_max - x_min)**2 + (y_max - y_min)**2))

    # Run the Monte Carlo simulation to get an ensemble of probable
    # lists of points included in each time slice.
    num_iterations = 200


    # set consistent pairwise bandwidth (binning of distances)
    use_kde = True
```

```python
pair_bw = None
kde_sample_n = 50
kde_custom=cuml_kde

# Run the Monte Carlo simulation to get an ensemble of probable
# lists of points included in each time slice.
simulations = clustering.mc_samples(doomsday_points,
                                    time_slices=time_slices,
                                    num_iterations=num_iterations)

# set consistent pairwise bandwidth (binning of distances)
# same as before with Angkor data
# Produce pairwise distances to explore clustering structure
pairwise_density, support = clustering.temporal_pairwise(
    simulations,
    time_slices,
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n = kde_sample_n,
    max_distance = max_distance,
    kde_custom = kde_custom
)

# Get MC iterations for incorporating chronological uncertainty with BISE
bise_simulations = clustering.mc_samples(
    doomsday_points,
    time_slices,
    num_iterations=num_iterations,
    null_model=clustering.bise
)

# Calulate the pairwise distances for the LISE sample
bise_pairwise_density, bise_support = clustering.temporal_pairwise(
    bise_simulations,
    time_slices,
    bw = pair_bw,
    use_kde = use_kde,
    kde_sample_n=kde_sample_n,
    max_distance = max_distance,
    kde_custom=kde_custom
)
```

```
# Calculate the p-values for density differences between the observed
 ↪  points and
# the simulated CSR baseline per distance and temporal slice
p_diff_array, diff_array = clustering.p_diff(
    pairwise_density,
    bise_pairwise_density
)

time_slice_idx = np.where(time_slices == 1066)[0][0]

# List of density arrays
density_arrays = [diff_array]

# Generate the plot and get the figure and axis objects
fig, ax = plot_pdd(
    time_slices=time_slices,
    time_slice_idx=time_slice_idx,
    support=support,
    density_arrays=density_arrays,
    quantiles=[0.025, 0.975],
    density_names=["Diff Array"],
    colors=["blue"]
)

# Add a horizontal line at y=0
ax.axhline(y=0, color='red', linestyle='--', linewidth=1.5)

# Set title to county label
ax.set_title(f"PDD {j}")

# Get current tick positions and convert labels to km
x_ticks = ax.get_xticks()
ax.set_xticklabels(np.round(x_ticks / 1000, 1))  # e.g. 1000 → 1.0 km

# Update axis label
ax.set_xlabel("Distance (km)")

# Save the plot to the Output directory
output_path = os.path.join(output_dir, f"pdd_{j}.png")

plt.savefig(output_path, dpi=300, bbox_inches='tight')
```

```python
    # Close the plot to free memory
    plt.close(fig)
```

```
Processing Counties: 100%|     | 34/34 [1:11:27<00:00, 126.10s/it]
```

Density

In order to make high-level comparisons of population density between Hampshire and Angkor, we needed to derive reasonable estiamtes of population size and total area for Hampshire (for Angkor, recent published research could be used for rough estimates). On researching the topic, there turned out to be several different estimates for relevant parameters and so the following is intended to produce a simple, but defensible estimate of population density that accounts for uncertainty in the relevant parameters.

Rationale

The underlying Domesday evidence is incomplete and population estimates have to be inferred/reconstructed. Household counts are recorded through social and fiscal categories rather than as a direct census, and converting these counts into population requires assumptions about persons per household. In addition, density estimation requires an area denominator, and even that is not entirely fixed because Hampshire's historical extent can vary slightly depending on the cartographic or administrative source used. Since the comparison in the paper concerns the broader Hampshire settlement system, the analysis here treats total Hampshire population over total Hampshire area, including the Isle of Wight.

Accordingly, this notebook models Hampshire density as a derived random quantity:

$$\rho_H = \frac{P_H}{A_H}$$

where:

- P_H is total Hampshire population in 1086,
- A_H is Hampshire land area under the historical boundary definition used here,
- ρ_H is population density in persons per km^2.

Population model

Population is estimated from Domesday household counts using a mean household size parameter:

$$P_H = N_H \times s$$

where:

- N_H is the relevant Domesday household count or count-like population unit,
- s is the mean number of persons per household.

Household Size

The key uncertainty lies in s. Following La Poutré (2023), the limited medieval English evidence suggests household sizes around 5.3, 5.8, and 6.3 persons per household in later medieval contexts, but La Poutré argues that Domesday household size was likely smaller, highlighting Darby's suggested range of roughly 4–5 persons per household for 1086. This notebook therefore treats household size probabilistically, centering the prior in a Domesday-appropriate range while allowing some uncertainty around it informed by the variability in estimates reported by La Poutré.

La Poutré reviews the limited quantitative evidence available for household size in medieval England and discusses several reconstructed estimates derived from manorial demographic records. The estimates below are average persons per household.

Estimate	Source	Context	Page
~5.3	Hallam (1958)	Derived from manorial demographic data after correcting Hallam's family-size figures to include all offspring and adjusting for widows/widowers	p. 92
~6.3	Howell (1983)	Estimated from a 1280 tithing list for Kibworth Harcourt that records adult males; the population is reconstructed by doubling for females and adding children (<12 years)	p. 92
~5.8	La Poutré calculation	Derived from Glastonbury Abbey estates by combining customary tenants and landless sons and dividing estimated population by holdings	p. 93

La Poutré notes that these estimates cluster around a mean of roughly 5.8 persons per household for the late-thirteenth-century medieval demographic peak. However, he argues that household size in 1086 (Domesday) was likely smaller, because the late-medieval constraints on marriage and land access that inflated household size had not yet intensified. He therefore considers values below the thirteenth-century peak plausible and, so, refers favourably to Darby's suggested Domesday range of roughly 4–5 persons per household.

Area model

The area denominator is treated as fixed in the simplest version of the analysis, but the notebook is structured so that it can also be treated as uncertain if needed for future studies. This matters because historical Hampshire boundaries can differ slightly across modern GIS layers, historical county maps, and other reconstructed sources.

The area figures for Hampshire Ancient County derive from two 19th-century parliamentary census publications. The 1831 figure (1,018,550 acres) comes from the Enumeration Abstract (1833), a return compiled under an Act of the eleventh year of George IV's reign [1]. The 1851 figure (1,077,164 acres) comes from the Tables of Population and Houses (1851), which covered the divisions, registration counties, and districts of England and Wales [2]. Both figures were accessed via the GB Historical GIS / University of Portsmouth dataset (Vision of Britain) [3]. We also included an area estimate derived from a bounding hull polygon over all of the spatialized manorial estate locations extracted from the Domesday survey by the University of Hull and the National Archives.

Year	Acres	km²	Source
1831	1,018,550	4,122	*Enumeration Abstract*, BPP 1833 xxxvi–xxxviii, Command No. 149 [1]
1851	1,077,164	4,359	*Tables of Population and Houses*, BPP 1851 xliii, 73, Command No. 1339 [2]
2026	...	5041	Estimated in QGIS with a bounding convex hull on the manorial estate point data

Bibliography [1] Great Britain. Abstracts of the Answers and Returns: Enumeration. Command No. 149. British Parliamentary Papers 1833 xxxvi–xxxviii. London: HMSO, 1833.

[2] Great Britain. Tables of Population and Houses in the Divisions, Registration Counties, and Districts of England and Wales. Command No. 1339. British Parliamentary Papers 1851 xliii, 73. London: HMSO, 1851.

[3] GB Historical GIS / University of Portsmouth. "Hampshire Ancient County through Time: Historical Statistics on Population, Area (acres)." A Vision of Britain through Time. Accessed March 9, 2026. https://www.visionofbritain.org.uk/unit/10204434/cube/AREA_ACRES.

```python
# -----------------------------
# Data inputs
# -----------------------------

# Domesday Hampshire households from La Poutré (2023), chapter VIII
hampshire_rural_households = 10194
hampshire_urban_households = 1595
hampshire_total_households = hampshire_rural_households +
 ↪  hampshire_urban_households

# Hampshire land area estimates in km^2, including the Isle of Wight
# Historical estimate range + convex hull first-pass estimate
hampshire_area_estimates_km2 = np.array([4122, 4359, 5041], dtype=float)

# Historically derived / reconstructed household-size means (persons per
 ↪  household)
# from La Poutré (2023)
household_size_estimates = np.array([5.3, 6.3, 5.8], dtype=float)

# Domesday-oriented reference range noted by La Poutré following Darby
darby_domesday_range = (4.0, 5.0)

print("Rural households:", hampshire_rural_households)
print("Urban households:", hampshire_urban_households)
print("Total households:", hampshire_total_households)
print("Hampshire area estimates (km^2):", hampshire_area_estimates_km2)
print("Historical / reconstructed household-size means:",
 ↪  household_size_estimates)
print("Darby Domesday reference range:", darby_domesday_range)
```

```
Rural households: 10194
Urban households: 1595
Total households: 11789
Hampshire area estimates (km^2): [4122. 4359. 5041.]
Historical / reconstructed household-size means: [5.3 6.3 5.8]
Darby Domesday reference range: (4.0, 5.0)
```

And, with the available data collated, we can now estimate the population density of Late Anglo Saxon Hampshire with a Bayesian model:

```python
with pm.Model() as hampshire_density_model:
    # Fixed data
    n_households = pm.Data("n_households", hampshire_total_households)
    area_obs_km2 = pm.Data("area_obs_km2", hampshire_area_estimates_km2)
    hh_means_obs = pm.Data("hh_means_obs", household_size_estimates)

    # Domesday mean household size - prior centred on Darby's range
    mu_domesday = pm.Normal(
        "mu_domesday_household_size",
        mu=4.5,
        sigma=0.5,
    )

    # The three literature estimates are noisy observations of this mean
    sigma_recon = pm.Exponential("sigma_recon", lam=2)
    pm.Normal(
        "observed_means",
        mu=mu_domesday,
        sigma=sigma_recon,
        observed=hh_means_obs,
    )

    # Latent historical Hampshire area (km^2)
    # Prior centered on the middle estimate with fairly weak uncertainty
    historical_area_km2 = pm.Normal(
        "historical_area_km2",
        mu=4359,
        sigma=500,
    )

    # Observed area estimates as noisy measurements of the latent historical
    ↪ area
    sigma_area = pm.Exponential("sigma_area", lam=1/300)
    pm.Normal(
        "observed_area_estimates",
        mu=historical_area_km2,
        sigma=sigma_area,
        observed=area_obs_km2,
    )

    # Expected population
    lambda_population = pm.Deterministic(
```

```
        "lambda_population",
        n_households * mu_domesday,
    )

    # Total population count: Normal approximation to the sum of n_households
    # Poisson(mu_domesday) random variables. By CLT, sum ~ Normal(n*mu,
     ↪  sqrt(n*mu)).
    # Poisson variance is the minimum variance assumption consistent with a
    # counting distribution given only knowledge of the mean; no variance
    # information exists in the literature to justify a stronger assumption.
    total_population = pm.Normal(
        "total_population",
        mu=lambda_population,
        sigma=pt.sqrt(lambda_population),
    )

    density_per_km2 = pm.Deterministic(
        "density_per_km2",
        total_population / historical_area_km2,
    )

    idata = pm.sample(
        draws=4000,
        tune=2000,
        chains=4,
        target_accept=0.95,
        random_seed=42,
        progressbar=False,
    )
```

And we can summarize the results:

```
# Summary table
summary = az.summary(
    idata,
    var_names=[
        "mu_domesday_household_size",
        "sigma_recon",
        "historical_area_km2",
        "sigma_area",
        "lambda_population",
```

```
        "total_population",
        "density_per_km2",
    ],
    hdi_prob=0.95,
)

summary = summary[["mean", "sd", "hdi_2.5%", "hdi_97.5%"]]
summary
```

	mean	sd	hdi_2.5%	hdi_97.5%
mu_domesday_household_size	5.272	0.411	4.430	5.979
sigma_recon	0.774	0.399	0.215	1.556
historical_area_km2	4467.907	238.722	3981.388	4942.819
sigma_area	489.371	203.590	197.933	892.612
lambda_population	62150.870	4844.941	52224.488	70491.903
total_population	62149.613	4853.798	52010.689	70366.709
density_per_km2	13.951	1.332	11.118	16.410